

ICPC Notebook Java

A progressive team reference for Java 17

A progressive algorithm reference for team practice

Profile: Advanced | 54 entries

Contents

How to use this reference **1**
Learning path . . . 1
Choosing an algorithm . . . 1
Java contest checklist . . . 2

Problem-solving foundations **2**

Guided problems **3**

1 Contest setup **4**
1.1 Standalone Main template . . . 4
1.2 Reusable fast input . . . 5

2 Fundamentals **5**
2.1 Loops, arrays, prefix sums, and brute force . . . 6
2.2 Maps, sets, and stacks . . . 7
2.3 Recursion and exhaustive subsets . . . 8
2.4 Arrays and two pointers . . . 8
2.5 Binary search and bounds . . . 10
2.6 Interval scheduling and covering . . . 11

3 Range queries and data structures **12**
3.1 DSU, Fenwick, and segment tree . . . 12
3.2 2D prefix sums, sparse table, lazy tree . . . 14
3.3 Rollback disjoint set union . . . 15
3.4 Two-dimensional Fenwick tree . . . 16
3.5 Mo offline distinct queries . . . 17
3.6 Li Chao line container . . . 18

4 Dynamic programming **19**
4.1 First dynamic programs . . . 19
4.2 Classic DP patterns . . . 20
4.3 Subset, interval, scheduling, digit DP . . . 22
4.4 Knuth optimized adjacent merging . . . 23

5 Graphs **24**
5.1 Adjacency lists, BFS, and grid flood fill . . . 24
5.2 Traversal, shortest paths, MST . . . 25
5.3 Components, bipartite, 0-1 BFS, Floyd, Euler . . . 27
5.4 SCC, Bellman-Ford, LCA, max flow . . . 29
5.5 Bridges and articulation points . . . 32
5.6 2-SAT . . . 33
5.7 Minimum-cost flow . . . 33
5.8 Bipartite matching and vertex cover . . . 35
5.9 Global minimum cut . . . 36
5.10 Exact spanning-tree count . . . 37
5.11 Minimum-cost assignment . . . 38
5.12 Binary submodular energy minimization . . . 39

6 Trees **41**
6.1 Diameter, Euler tour, heavy-light . . . 42

7 Strings **43**
7.1 Characters, frequencies, and direct matching . . . 43
7.2 KMP, Manacher, Boyer-Moore, Z . . . 44
7.3 Trie and rolling hash . . . 46
7.4 Suffix array and LCP . . . 47
7.5 Aho-Corasick . . . 48
7.6 Minimum cyclic rotation . . . 49
7.7 Suffix automaton . . . 50

8 Number theory and combinatorics **51**
8.1 GCD, trial division, divisors, and powers . . . 52
8.2 GCD, modular arithmetic, primality, CRT . . . 53
8.3 Factors, phi, inverse, modular nCr, matrices . . . 55
8.4 Exact binomial and Catalan numbers . . . 57
8.5 Discrete log, modular root, Mobius . . . 57
8.6 Derangements, Stirling, partitions, Burnside . . . 59

9 Polynomial methods **60**
9.1 Number-theoretic transform . . . 60

10 Numerical methods **61**
10.1 Gaussian elimination and determinant . . . 61
10.2 Subset zeta, Mobius, and XOR transforms . . . 63
10.3 Fast Fourier transform convolution . . . 64

11 Geometry **65**
11.1 Distance, polygons, hull, closest pair . . . 65
11.2 Exact orientation and intersection . . . 67
11.3 Lines, segments, and circle intersections . . . 68
11.4 Two-dimensional KD-tree nearest neighbor . . . 69
11.5 3D vectors and spherical distance . . . 70

12 Game theory **71**
12.1 Nim, mex, Grundy, minimax . . . 71

How to use this reference

Read the algorithm's use case, cost, and caution before copying its code. The listing is the same Java source compiled by the repository tests. Use the exporter to combine the selected classes into one judge-ready Main.java.

Learning path: Advanced

Who this is for Teams preparing for problems that combine several techniques or require specialized algorithms.
Before starting The intermediate reference, graph traversal, DP, modular arithmetic, and proof by invariant.
Practice method For each advanced method, write the invariant and construct a small case that breaks an incorrect implementation.

Learning goals

- Recognize reductions to flows, satisfiability, tree paths, and convolution.
- Check hidden assumptions such as negative costs and recursion depth.
- Estimate both asymptotic cost and Java memory overhead.
- Validate complex algorithms with adversarial and differential tests.
- Check specialized data structures and numerical preconditions before use.
- Model submodular binary energies as graph cuts and select FFT or NTT by precision needs.

Conventions

- Vertices and arrays are zero-indexed.
- Ranges are half-open [left, right) unless documented otherwise.
- Check overflow, input bounds, and recursion depth before using a snippet.
- Choose the simplest algorithm that fits the problem constraints.

Choosing an algorithm

Problem signal	Choice	Typical cost
Small sequence, simple condition	Direct scan or brute force	O(n) or O(n^2)
Frequency or membership lookup	HashMap or HashSet	Expected O(1) per operation
All subsets of a small set	Backtracking	O(2^n)
Static interval sums	Prefix sums	O(n) build, O(1) query
Point updates and interval sums	Fenwick or segment tree	O(log n) per operation
Range additions and interval sums	Lazy segment tree	O(log n) per operation
Unweighted graph or grid	Breadth-first search	O(V+E)
Incremental connectivity	Disjoint set union	Almost O(1) amortized
Unweighted, 0/1, nonnegative paths	BFS, 0-1 BFS, Dijkstra	O(V+E) to O((V+E) log V)
Negative edges or all-pairs paths	Bellman-Ford, Floyd-Warshall	O(VE) or O(V^3)
One pattern or many patterns	KMP, Z, Aho-Corasick	Linear in input size
Changing tree path sums	Heavy-light decomposition	O(log^2 n) per query
Subset states	Bitmask dynamic programming	Often O(n^2 2^n)
Large polynomial product	Number-theoretic transform	O(n log n)
Many static distinct range queries	Mo ordering	Typical O((n+q) sqrt n)
Undo connectivity merges	Rollback DSU	O(log n) union/find
Assign left items to right items	Hopcroft-Karp matching	O(E sqrt V)

Problem signal	Choice	Typical cost
Minimum of changing lines	Li Chao tree	O(log domain)
Dense linear equations	Gaussian elimination	O(n^3)
Dense minimum-cost assignment	Hungarian algorithm	O(n^2 m)
Binary pairwise energy with submodular costs	s-t graph cut	Max-flow dependent
Large polynomial product with safe rounding	FFT convolution	O(n log n)
Repeated nearest-neighbor queries in static 2D points	KD-tree	Average O(log n)

Java contest checklist

- Check time and memory limits before choosing object-heavy structures.
- Use long for sums, distances, products, and counts that may exceed int.
- Check whether input is ASCII tokens or requires Unicode handling.
- Watch recursion depth on long paths; use an explicit stack if needed.
- Confirm whether edges are directed, whether weights can be negative, and whether ranges are inclusive.
- Test empty, singleton, duplicate, disconnected, and boundary cases.

Problem-solving foundations

Recognize the reduction

Advanced problems often hide a standard model. Decide whether the objects are vertices, intervals, states, Boolean literals, flow units, or polynomial coefficients.

Example Choosing compatible assignments with pairwise either-or constraints may become 2-SAT; routing units at minimum total cost may become min-cost flow.
Try it Describe the model and prove each problem constraint is represented exactly once.

Budget both asymptotics and memory

In Java, an O(n^2) table may be limited by object headers and array references as well as cell count. Recursive graph traversals may hit the call stack before time becomes a problem.

Example A 2^20 by 20 long table contains about 160 MiB of values before array overhead.
Try it Estimate bytes and calls for the largest case before selecting an implementation.

Validate the difficult invariant

Use a slower oracle on small inputs, preserve failing seeds, and include adversarial cases. State preconditions such as nonnegative initial costs, prime modulus, and coordinate range.

Example Compare an NTT product with O(nm) multiplication for random short polynomials.
Try it Construct a case with duplicates, disconnected components, or a later residual reroute.

Guided problems

Translate clauses into implications

Task Find values of a and b satisfying (a or b), (not a or b), and (not b or a).
Result a = true and b = true is a solution
Reasoning For each clause x or y, add not x to y and not y to x. Solve the implication graph by SCC and reject if a variable meets its negation.
Use TwoSat
Cost O(variables + clauses).
Check Enumerate all four assignments to confirm the returned solution satisfies every clause.

Multiply two polynomials

Task Multiply coefficient arrays [1,2] and [3,4] modulo 998244353.
Result [3,10,8]
Reasoning The coefficients come from convolution: 1*3, then 1*4 + 2*3, then 2*4. NTT makes the same operation efficient for long arrays.
Use Convolution
Cost O(n log n) for a padded transform of length n.
Check Compare random short arrays against nested-loop multiplication.

Route flow at minimum cost

Task Send two units from 0 to 3. Edges 0-1 and 1-3 have cost 1 and capacity 1; edges 0-2 and 2-3 have costs 2 and 1 with capacity 1.
Result Flow 2 with total cost 5
Reasoning One unit takes 0-1-3 at cost 2. The other takes 0-2-3 at cost 3. Residual reverse edges allow later augmentations to revise a choice.
Use MinCostFlow
Cost Depends on the number of augmenting paths and shortest-path work.
Check Verify delivered flow, capacity limits, and the sum of used edge costs.

1 Contest setup

Start with a compilable single-file submission and fast input.

1.1 Standalone Main template

Level Beginner
Use when Start a Java judge submission.
Cost Linear in the input size.
Watch for Use the exporter to include required helper classes.

Core idea. Start from a working input/output skeleton so effort stays on the algorithm.
Worked example. Read two integers, add them, and print one line.
Practice. Compile the template before inserting helper classes.

```
import java.io.*;
import java.util.*;

/** Standalone template for judges that require a single Main.java file. */
public class Main {
    static final class FastScanner {
        private final InputStream in = new BufferedInputStream(System.in);
        private final byte[] buffer = new byte[1 << 16];
        private int pos, len;

        private int read() throws IOException {
            if (pos >= len) {
                len = in.read(buffer);
                pos = 0;
                if (len < 0) return -1;
            }
            return buffer[pos++] & 0xff;
        }

        String next() throws IOException {
            int c;
            do { c = read(); } while (c >= 0 && c <= ' ');
            if (c < 0) return null;
            StringBuilder token = new StringBuilder();
            while (c > ' ') { token.append((char) c); c = read(); }
            return token.toString();
        }

        int nextInt() throws IOException { return Integer.parseInt(next()); }
        long nextLong() throws IOException { return Long.parseLong(next()); }
    }

    public static void main(String[] args) throws Exception {
        FastScanner in = new FastScanner();
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        // Solve the problem here. Read with in.nextInt(), write with out.println().
        out.flush();
    }
}
```

1.2 Reusable fast input

Level Beginner
Use when Read large ASCII-token input.
Cost Linear time; fixed-size buffer.
Watch for Not a Unicode decoder.

Core idea. Buffered token input reduces per-character overhead on large files.
Worked example. Read n, then process exactly n integers with nextInt().
Practice. Test EOF, negative values, and the final token without a newline.

```
package notebook;

import java.io.BufferedInputStream;
import java.io.IOException;
import java.io.InputStream;

/** Fast byte input for ASCII contest tokens. Use PrintWriter/StringBuilder for output. */
public final class ContestIO {
    private final InputStream in;
    private final byte[] buffer = new byte[1 << 16];
    private int pos, len;

    public ContestIO(InputStream input) { in = new BufferedInputStream(input); }

    private int read() throws IOException {
        if (pos >= len) {
            len = in.read(buffer);
            pos = 0;
            if (len < 0) return -1;
        }
        return buffer[pos++] & 0xff;
    }

    /** Returns null at EOF. Tokens are decoded as ASCII. */
    public String next() throws IOException {
        int c;
        do { c = read(); } while (c >= 0 && c <= ' ');
        if (c < 0) return null;
        StringBuilder token = new StringBuilder();
        while (c > ' ') { token.append((char) c); c = read(); }
        return token.toString();
    }

    public int nextInt() throws IOException { return Integer.parseInt(next()); }
    public long nextLong() throws IOException { return Long.parseLong(next()); }
}
```

2 Fundamentals

Arrays, windows, sorting, and binary search come first.

2.1 Loops, arrays, prefix sums, and brute force

Level Beginner
Use when Process a sequence or test all small pairs.
Cost O(n) scans; O(n^2) pair search.
Watch for A prefix array has length n+1 and answers half-open ranges.

Core idea. A direct scan is often the clearest baseline. Prefix sums store the total before each index so one subtraction answers a range.
Worked example. For [3, -2, 5], prefix = [0, 3, 1, 6] and sum [1, 3) = 6 - 3 = 3.
Practice. Compare every range sum against a loop on small random arrays.

```
package notebook;

import java.util.Arrays;

/** Small array patterns that are useful before learning advanced data structures. */
public final class BeginnerArrays {
    private BeginnerArrays() {}

    /** Sum all values. O(n) time, O(1) extra space; throws if the sum exceeds long. */
    public static long sum(int[] values) {
        long result = 0;
        for (int value : values) result = Math.addExact(result, value);
        return result;
    }

    /** Return the index of the first target, or -1. O(n) time, O(1) space. */
    public static int linearSearch(int[] values, int target) {
        for (int i = 0; i < values.length; i++) if (values[i] == target) return i;
        return -1;
    }

    /** Count occurrences of a target. O(n) time, O(1) space. */
    public static int count(int[] values, int target) {
        int result = 0;
        for (int value : values) if (value == target) result++;
        return result;
    }

    /** Reverse an array in place. O(n) time, O(1) extra space. */
    public static void reverse(int[] values) {
        for (int left = 0, right = values.length - 1; left < right; left++, right--) {
            int temp = values[left]; values[left] = values[right]; values[right] = temp;
        }
    }

    /** Inclusive prefix sums with prefix[0] = 0. O(n) time and space. */
    public static long[] prefixSums(int[] values) {
        long[] prefix = new long[values.length + 1];
        for (int i = 0; i < values.length; i++) prefix[i + 1] = Math.addExact(prefix[i], values[i]);
        return prefix;
    }

    /** Sum [left, right) using a prefix array. O(1) time. */
    public static long rangeSum(long[] prefix, int left, int right) {
        if (left < 0 || right < left || right >= prefix.length) throw new IndexOutOfBoundsException();
        return prefix[right] - prefix[left];
    }

    /** Two-sum by trying every pair; returns indices or an empty array. O(n^2) time. */
    public static int[] pairWithSum(int[] values, long target) {
        for (int i = 0; i < values.length; i++)
            for (int j = i + 1; j < values.length; j++)
                if ((long) values[i] + values[j] == target) return new int[] {i, j};
        return new int[0];
    }

    /** Copy and sort values without modifying the input. O(n log n) time. */
```

```
    public static int[] sortedCopy(int[] values) {
        int[] result = values.clone();
        Arrays.sort(result);
        return result;
    }
}
```

2.2 Maps, sets, and stacks

Level Beginner
Use when Count frequencies, remove duplicates, or check nested delimiters.
Cost Expected O(n) time for hash collections; O(n) stack space.
Watch for HashMap and HashSet have no guaranteed iteration order.

Core idea. A map stores a value for each key, a set stores membership, and a stack reverses the order in which items are processed.
Worked example. For [4,2,4], the frequency map stores 4 to 2 and 2 to 1. The string ([{}]) is balanced.
Practice. Solve a frequency-count problem with both a map and a sorted array, then compare the tradeoffs.

```
package notebook;

import java.util.ArrayDeque;
import java.util.HashMap;
import java.util.HashSet;
import java.util.Map;
import java.util.Set;

/** Everyday Java collections: map, set, and stack. */
public final class BeginnerCollections {
    private BeginnerCollections() {}

    /** Count each integer with a hash map. Expected O(n) time and O(distinct) space. */
    public static Map<Integer, Integer> frequencies(int[] values) {
        Map<Integer, Integer> result = new HashMap<>();
        for (int value : values) result.merge(value, 1, Integer::sum);
        return result;
    }

    /** Count distinct integers with a hash set. Expected O(n) time and space. */
    public static int distinctCount(int[] values) {
        Set<Integer> seen = new HashSet<>();
        for (int value : values) seen.add(value);
        return seen.size();
    }

    /** Check balanced (), [], and {} brackets. O(n) time, O(n) stack space. */
    public static boolean balancedBrackets(String text) {
        ArrayDeque<Character> stack = new ArrayDeque<>();
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (ch == '(' || ch == '[' || ch == '{') stack.push(ch);
            else if (ch == ')' || ch == ']' || ch == '}') {
                if (stack.isEmpty()) return false;
                char open = stack.pop();
                if (ch == ')' && open != '(' || ch == ']' && open != '[' || ch == '}' && open != '{')
                    return false;
            } else throw new IllegalArgumentException("expected brackets only");
        }
        return stack.isEmpty();
    }
}
```

2.3 Recursion and exhaustive subsets

Level Beginner
Use when Enumerate all choices when n is small.
Cost O(n) factorial; O(2^n) subset search.
Watch for Exponential search is only practical for small n; recursion uses stack space.

Core idea. A recursive function solves a smaller instance, reaches a base case, then combines answers. For each subset element, branch into skip and take.
Worked example. For [1,2,3] and target 3, subsets [3] and [1,2] both match.
Practice. Draw the recursion tree for three values and mark its base cases.

```
package notebook;

/** Recursion and exhaustive search when the input is small enough. */
public final class BeginnerBacktracking {
    private BeginnerBacktracking() {}

    /** Factorial using a smaller subproblem. O(n) time and stack; n must be in [0,20]. */
    public static long factorial(int n) {
        if (n < 0 || n > 20) throw new IllegalArgumentException("n must be in [0,20]");
        if (n <= 1) return 1;
        return n * factorial(n - 1);
    }

    /** Count subsets whose sum equals target. O(2^n) time, O(n) stack; n <= 25. */
    public static long countSubsetsWithSum(int[] values, long target) {
        if (values.length > 25) throw new IllegalArgumentException("too many subsets");
        return count(values, 0, 0, target);
    }

    private static long count(int[] values, int index, long sum, long target) {
        if (index == values.length) return sum == target ? 1 : 0;
        return count(values, index + 1, sum, target)
                + count(values, index + 1, Math.addExact(sum, values[index]), target);
    }
}
```

2.4 Arrays and two pointers

Level Intermediate
Use when Use prefix sums, differences, compression, windows, inversions, or permutations.
Cost O(n) to O(n log n).
Watch for Sliding-window sums require nonnegative values.

Core idea. Prefix, window, permutation, and inversion patterns replace repeated work with maintained state.
Worked example. A nonnegative window can grow right and shrink left while its sum is too large.
Practice. State the invariant before moving either pointer.

```
package notebook;

import java.util.Arrays;

/** Building blocks used before choosing a more specialized algorithm. */
public final class Fundamentals {
    private Fundamentals() {}

    /** prefix[i] is the sum of a[0..i); query [l,r) as prefix[r]-prefix[l]. O(n). */
    public static long[] prefixSums(int[] a) {
        long[] prefix = new long[a.length + 1];
        for (int i = 0; i < a.length; i++) prefix[i + 1] = prefix[i] + a[i];
        return prefix;
    }

    /** Apply [l,r) additions in O(updates+n). Each update is {l,r,delta}. */
```

```
public static long[] rangeAdd(int n, long[][] updates) {
    long[] difference = new long[n + 1];
    for (long[] update : updates) {
        int l = Math.toIntExact(update[0]), r = Math.toIntExact(update[1]);
        if (l < 0 || l > r || r > n) throw new IndexOutOfBoundsException();
        difference[l] += update[2];
        difference[r] -= update[2];
    }
    long[] result = new long[n];
    long sum = 0;
    for (int i = 0; i < n; i++) { sum += difference[i]; result[i] = sum; }
    return result;
}

/** Sorted distinct values. Use Arrays.binarySearch(result, x) for compressed IDs. O(n log n). */
public static int[] sortedUnique(int[] a) {
    int[] sorted = a.clone();
    Arrays.sort(sorted);
    int size = 0;
    for (int value : sorted)
        if (size == 0 || value != sorted[size - 1]) sorted[size++] = value;
    return Arrays.copyOf(sorted, size);
}

/** Two pointers: number of pairs i<j with a[i]+a[j] <= limit. Input must be sorted. O(n). */
public static long countPairsAtMost(int[] sorted, long limit) {
    long count = 0;
    for (int l = 0, r = sorted.length - 1; l < r;) {
        if ((long) sorted[l] + sorted[r] <= limit) { count += r - l; l++; }
        else r--;
    }
    return count;
}

/** Longest window with sum <= limit; requires nonnegative elements. O(n). */
public static int longestWindowAtMost(int[] a, long limit) {
    long sum = 0;
    int best = 0, left = 0;
    for (int right = 0; right < a.length; right++) {
        if (a[right] < 0) throw new IllegalArgumentException("negative element");
        sum += a[right];
        while (left <= right && sum > limit) sum -= a[left++];
        best = Math.max(best, right - left + 1);
    }
    return best;
}

/** Maximum nonempty subarray sum. O(n); returns 0 for empty input. */
public static long maxSubarraySum(int[] a) {
    if (a.length == 0) return 0;
    long ending = a[0], best = ending;
    for (int i = 1; i < a.length; i++) {
        ending = Math.max(a[i], ending + a[i]);
        best = Math.max(best, ending);
    }
    return best;
}

/** Number of index pairs i<j with a[i]>a[j]. O(n log n). */
public static long inversions(int[] a) {
    int[] copy = a.clone(), scratch = new int[a.length];
    return mergeCount(copy, scratch, 0, a.length);
}

private static long mergeCount(int[] a, int[] tmp, int lo, int hi) {
    if (hi - lo <= 1) return 0;
    int mid = (lo + hi) >>> 1;
    long count = mergeCount(a, tmp, lo, mid) + mergeCount(a, tmp, mid, hi);
    int i = lo, j = mid;
    for (int k = lo; k < hi; k++) {
        if (j == hi || (i < mid && a[i] <= a[j])) tmp[k] = a[i++];
        else { tmp[k] = a[j++]; count += mid - i; }
```

```
        }
        System.arraycopy(tmp, lo, a, lo, hi - lo);
        return count;
    }

    /** Lexicographically next permutation in place; false after wrapping to sorted order. O(n). */
    public static boolean nextPermutation(int[] a) {
        int i = a.length - 2;
        while (i >= 0 && a[i] >= a[i + 1]) i--;
        if (i >= 0) {
            int j = a.length - 1;
            while (a[j] <= a[i]) j--;
            int t = a[i]; a[i] = a[j]; a[j] = t;
        }
        for (int l = i + 1, r = a.length - 1; l < r; l++, r--) {
            int t = a[l]; a[l] = a[r]; a[r] = t;
        }
        return i >= 0;
    }

    /** Enumerate nonempty submasks of mask; mask must be nonnegative. O(2^popcount(mask)). */
    public static int[] nonemptySubmasks(int mask) {
        if (mask < 0) throw new IllegalArgumentException();
        if (Integer.bitCount(mask) > 20) throw new IllegalArgumentException("too many submasks to materialize");
        int[] result = new int[(1 << Integer.bitCount(mask)) - 1];
        int index = 0;
        for (int sub = mask; sub != 0; sub = (sub - 1) & mask) result[index++] = sub;
        return result;
    }
}
```

2.5 Binary search and bounds

Level Beginner
Use when Search sorted data or a monotone answer predicate.
Cost O(log n) search.
Watch for Sorted input and monotonicity are preconditions.

Core idea. Binary search keeps an interval that is guaranteed to contain the answer.
Worked example. For sorted [1, 2, 2, 5], lowerBound(2) is 1 and upperBound(2) is 3.
Practice. Test an empty array and targets below and above every element.

```
package notebook;

import java.util.function.LongPredicate;

/** Search and sorting routines. Input to searches must be sorted ascending. */
public final class SearchingSorting {
    private SearchingSorting() {}

    /** O(log n); returns any matching index, or -1. */
    public static int binarySearch(int[] a, int key) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == key) return mid;
            if (a[mid] < key) lo = mid + 1;
            else hi = mid - 1;
        }
        return -1;
    }

    /** O(log n); returns any matching index, or -1. */
    public static int ternarySearch(int[] a, int key) {
        int lo = 0, hi = a.length - 1;
        while (lo <= hi) {
            int m1 = lo + (hi - lo) / 3;
            int m2 = hi - (hi - lo) / 3;
```

```
            if (a[m1] == key) return m1;
            if (a[m2] == key) return m2;
            if (key < a[m1]) hi = m1 - 1;
            else if (key > a[m2]) lo = m2 + 1;
            else { lo = m1 + 1; hi = m2 - 1; }
        }
        return -1;
    }

    /** O(n + 65536) counting sort for Java UTF-16 char units. */
    public static void countingSort(char[] a) {
        int[] count = new int[Character.MAX_VALUE + 1];
        for (char c : a) count[c]++;
        int i = 0;
        for (int c = 0; c < count.length; c++) {
            while (count[c]-- > 0) a[i++] = (char) c;
        }
    }

    /** Lower bound: first index i with a[i] >= key. O(log n). */
    public static int lowerBound(int[] a, int key) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] < key) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }

    /** First index with a[i] > key in a sorted array. O(log n). */
    public static int upperBound(int[] a, int key) {
        int lo = 0, hi = a.length;
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] <= key) lo = mid + 1;
            else hi = mid;
        }
        return lo;
    }

    /** First x in nonnegative [low,high] satisfying a monotone false-then-true predicate. */
    public static long firstTrue(long low, long high, LongPredicate predicate) {
        if (low < 0 || low > high || !predicate.test(high)) throw new IllegalArgumentException("no true value in
     range");
        while (low < high) {
            long mid = low + ((high - low) >>> 1);
            if (predicate.test(mid)) high = mid;
            else low = mid + 1;
        }
        return low;
    }
}
```

2.6 Interval scheduling and covering

Level Beginner
Use when Select compatible intervals or cover intervals with few points.
Cost O(n log n) from sorting.
Watch for Scheduling uses half-open intervals; point covering uses closed intervals.

Core idea. Sort by earliest finishing time. Choosing an early finish leaves as much room as possible for later intervals.
Worked example. Among [0,3), [1,2), [2,4), choose the last two.
Practice. Try to prove the choice by exchanging an optimal first interval with the greedy one.

```
package notebook;

import java.util.Arrays;
```

```java
import java.util.Comparator;

/** Greedy choices with simple exchange-argument proofs. */
public final class BeginnerGreedy {
    private BeginnerGreedy() {}

    /** Maximum number of non-overlapping [start,end) intervals. O(n log n) time. */
    public static int maxNonOverlapping(long[][] intervals) {
        long[][] sorted = new long[intervals.length][];
        for (int i = 0; i < intervals.length; i++) {
            if (intervals[i].length != 2 || intervals[i][0] > intervals[i][1])
                throw new IllegalArgumentException("invalid interval");
            sorted[i] = intervals[i].clone();
        }
        Arrays.sort(sorted, Comparator.comparingLong((long[] interval) -> interval[1])
                .thenComparingLong(interval -> interval[0]));
        int chosen = 0;
        long lastEnd = Long.MIN_VALUE;
        for (long[] interval : sorted) if (interval[0] >= lastEnd) {
            chosen++;
            lastEnd = interval[1];
        }
        return chosen;
    }

    /** Minimum points that cover closed intervals [left,right]. O(n log n) time. */
    public static int pointsToCover(long[][] intervals) {
        long[][] sorted = new long[intervals.length][];
        for (int i = 0; i < intervals.length; i++) {
            if (intervals[i].length != 2 || intervals[i][0] > intervals[i][1])
                throw new IllegalArgumentException("invalid interval");
            sorted[i] = intervals[i].clone();
        }
        Arrays.sort(sorted, Comparator.comparingLong(interval -> interval[1]));
        int count = 0;
        long lastPoint = 0;
        boolean hasPoint = false;
        for (long[] interval : sorted) if (!hasPoint || interval[0] > lastPoint) {
            lastPoint = interval[1];
            hasPoint = true;
            count++;
        }
        return count;
    }
}
```

3 Range queries and data structures

Choose by update type, query type, and whether the data changes.

3.1 DSU, Fenwick, and segment tree

Level Intermediate
Use when Track connectivity, point updates, or range sums.
Cost Almost O(1) DSU; O(log n) range operations.
Watch for Sums use long.

Core idea. Use DSU for merging groups, Fenwick for prefix sums with updates, and segment trees for general ranges.
Worked example. After union(0,1), find(0) and find(1) represent the same component.
Practice. Compare random updates and queries against a plain array.

```java
package notebook;
```

```java
/** Zero-indexed contest data structures. */
public final class DataStructures {
    private DataStructures() {}

    /** Disjoint set union with path compression and union by size. */
    public static final class DSU {
        private final int[] parent, size;
        private int components;
        public DSU(int n) {
            if (n < 0) throw new IllegalArgumentException();
            parent = new int[n]; size = new int[n]; components = n;
            for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
        }
        public int find(int x) {
            if (parent[x] != x) parent[x] = find(parent[x]);
            return parent[x];
        }
        public boolean union(int a, int b) {
            a = find(a); b = find(b);
            if (a == b) return false;
            if (size[a] < size[b]) { int t = a; a = b; b = t; }
            parent[b] = a; size[a] += size[b]; components--;
            return true;
        }
        public int size(int x) { return size[find(x)]; }
        public int components() { return components; }
    }

    /** Fenwick tree for point additions and half-open range sums. O(log n) per operation. */
    public static final class Fenwick {
        private final long[] tree;
        public Fenwick(int n) {
            if (n < 0) throw new IllegalArgumentException();
            tree = new long[n + 1];
        }
        public void add(int index, long delta) {
            if (index < 0 || index >= tree.length - 1) throw new IndexOutOfBoundsException();
            for (int i = index + 1; i < tree.length; i += i & -i) tree[i] += delta;
        }
        public long prefixSum(int endExclusive) {
            if (endExclusive < 0 || endExclusive >= tree.length) throw new IndexOutOfBoundsException();
            long result = 0;
            for (int i = endExclusive; i > 0; i -= i & -i) result += tree[i];
            return result;
        }
        public long rangeSum(int left, int rightExclusive) {
            if (left > rightExclusive) throw new IllegalArgumentException();
            return prefixSum(rightExclusive) - prefixSum(left);
        }
    }

    /** Iterative segment tree for point assignment and half-open range sums. */
    public static final class SegmentTree {
        private final long[] tree;
        private final int n, base;
        public SegmentTree(long[] values) {
            n = values.length;
            int power = 1;
            while (power < n) power <<= 1;
            base = power;
            tree = new long[2 * base];
            for (int i = 0; i < n; i++) tree[base + i] = values[i];
            for (int i = base - 1; i > 0; i--) tree[i] = tree[2 * i] + tree[2 * i + 1];
        }
        public void set(int index, long value) {
            if (index < 0 || index >= n) throw new IndexOutOfBoundsException();
            int p = base + index;
            tree[p] = value;
            for (p >>= 1; p > 0; p >>= 1) tree[p] = tree[2 * p] + tree[2 * p + 1];
        }
        public long sum(int left, int rightExclusive) {
            if (left < 0 || left > rightExclusive || rightExclusive > n) throw new IndexOutOfBoundsException();
```

```
            long result = 0;
            for (int l = left + base, r = rightExclusive + base; l < r; l >>= 1, r >>= 1) {
                if ((l & 1) != 0) result += tree[l++];
                if ((r & 1) != 0) result += tree[--r];
            }
            return result;
        }
    }
}
```

3.2 2D prefix sums, sparse table, lazy tree

Level Intermediate
Use when Answer rectangle sums, immutable minima, or range additions.
Cost O(1) immutable queries; O(log n) lazy operations.
Watch for The sparse table supports idempotent minimum.

Core idea. Preprocessing trades build time and memory for faster repeated queries.
Worked example. A rectangle sum uses four 2D prefix values by inclusion-exclusion.
Practice. Check one-cell, whole-grid, and boundary rectangles.

```
package notebook;

/** Immutable and mutable range query structures; all ranges are half-open. */
public final class RangeQueries {
    private RangeQueries() {}

    /** Rectangle sums on [top,bottom) x [left,right). O(rows*cols) build, O(1) query. */
    public static final class Prefix2D {
        private final long[][] prefix;
        public Prefix2D(int[][] grid) {
            int rows = grid.length, cols = rows == 0 ? 0 : grid[0].length;
            prefix = new long[rows + 1][cols + 1];
            for (int r = 0; r < rows; r++) {
                if (grid[r].length != cols) throw new IllegalArgumentException("ragged grid");
                for (int c = 0; c < cols; c++)
                    prefix[r + 1][c + 1] = grid[r][c] + prefix[r][c + 1]
                        + prefix[r + 1][c] - prefix[r][c];
            }
        }
        public long sum(int top, int left, int bottom, int right) {
            return prefix[bottom][right] - prefix[top][right]
                - prefix[bottom][left] + prefix[top][left];
        }
    }

    /** Static range minimum. O(n log n) build, O(1) query, O(n log n) space. */
    public static final class SparseTableMin {
        private final int[][] table;
        private final int[] log;
        public SparseTableMin(int[] a) {
            log = new int[a.length + 1];
            for (int i = 2; i < log.length; i++) log[i] = log[i / 2] + 1;
            table = new int[a.length == 0 ? 0 : log[a.length] + 1][a.length];
            if (a.length == 0) return;
            table[0] = a.clone();
            for (int k = 1; k < table.length; k++)
                for (int i = 0; i + (1 << k) <= a.length; i++)
                    table[k][i] = Math.min(table[k - 1][i], table[k - 1][i + (1 << (k - 1))]);
        }
        public int min(int left, int right) {
            if (left < 0 || left >= right || right > log.length - 1) throw new IndexOutOfBoundsException();
            int k = log[right - left];
            return Math.min(table[k][left], table[k][right - (1 << k)]);
        }
    }

    /** Range addition and range sum. O(n) build, O(log n) operations. */
```

```
    public static final class LazySegmentTree {
        private final long[] sum, lazy;
        private final int n;
        public LazySegmentTree(long[] values) {
            n = values.length;
            sum = new long[4 * Math.max(1, n)];
            lazy = new long[sum.length];
            if (n > 0) build(1, 0, n, values);
        }
        private void build(int node, int l, int r, long[] values) {
            if (r - l == 1) { sum[node] = values[l]; return; }
            int mid = (l + r) >>> 1;
            build(node * 2, l, mid, values);
            build(node * 2 + 1, mid, r, values);
            sum[node] = sum[node * 2] + sum[node * 2 + 1];
        }
        private void apply(int node, int length, long delta) {
            sum[node] += delta * length;
            lazy[node] += delta;
        }
        private void push(int node, int l, int r) {
            if (lazy[node] == 0 || r - l <= 1) return;
            int mid = (l + r) >>> 1;
            apply(node * 2, mid - l, lazy[node]);
            apply(node * 2 + 1, r - mid, lazy[node]);
            lazy[node] = 0;
        }
        public void add(int left, int right, long delta) {
            if (left < 0 || left > right || right > n) throw new IndexOutOfBoundsException();
            if (n > 0) add(1, 0, n, left, right, delta);
        }
        private void add(int node, int l, int r, int ql, int qr, long delta) {
            if (qr <= l || r <= ql) return;
            if (ql <= l && r <= qr) { apply(node, r - l, delta); return; }
            push(node, l, r);
            int mid = (l + r) >>> 1;
            add(node * 2, l, mid, ql, qr, delta);
            add(node * 2 + 1, mid, r, ql, qr, delta);
            sum[node] = sum[node * 2] + sum[node * 2 + 1];
        }
        public long sum(int left, int right) {
            if (left < 0 || left > right || right > n) throw new IndexOutOfBoundsException();
            return n == 0 ? 0 : sum(1, 0, n, left, right);
        }
        private long sum(int node, int l, int r, int ql, int qr) {
            if (qr <= l || r <= ql) return 0;
            if (ql <= l && r <= qr) return sum[node];
            push(node, l, r);
            int mid = (l + r) >>> 1;
            return sum(node * 2, l, mid, ql, qr) + sum(node * 2 + 1, mid, r, ql, qr);
        }
    }
}
```

3.3 Rollback disjoint set union

Level Advanced
Use when Undo connectivity merges during offline processing.
Cost O(log n) find and union.
Watch for No path compression.

Core idea. Record each successful union so it can be reversed.
Worked example. Merge 0-1, snapshot, merge 1-2, then roll back to separate 2.
Practice. Compare snapshots against rebuilt components.

```
package notebook;

import java.util.ArrayList;
```

```java
import java.util.List;

/** Union by size without path compression, so earlier states can be restored. */
public final class RollbackDSU {
    private final int[] parent, size;
    private final List<int[]> history = new ArrayList<>();
    private int components;

    public RollbackDSU(int n) {
        if (n < 0) throw new IllegalArgumentException();
        parent = new int[n]; size = new int[n]; components = n;
        for (int i = 0; i < n; i++) { parent[i] = i; size[i] = 1; }
    }

    /** O(log n) worst case because union by size bounds tree height. */
    public int find(int vertex) {
        while (vertex != parent[vertex]) vertex = parent[vertex];
        return vertex;
    }

    /** Return whether two components merged. O(log n). */
    public boolean union(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (size[a] < size[b]) { int temp = a; a = b; b = temp; }
        history.add(new int[] {a, b, size[b]});
        parent[b] = a; size[a] += size[b]; components--;
        return true;
    }

    /** A token for a later rollback. O(1). */
    public int snapshot() { return history.size(); }

    /** Undo all successful unions after a snapshot. O(number of undone unions). */
    public void rollback(int snapshot) {
        if (snapshot < 0 || snapshot > history.size()) throw new IllegalArgumentException();
        while (history.size() > snapshot) {
            int[] change = history.remove(history.size() - 1);
            parent[change[1]] = change[1];
            size[change[0]] -= change[2];
            components++;
        }
    }

    public int components() { return components; }
    public int size(int vertex) { return size[find(vertex)]; }
}
```

3.4 Two-dimensional Fenwick tree

Level Intermediate
Use when Update cells and query rectangle sums.
Cost O(log rows * log columns).
Watch for Zero-based half-open rectangles.

Core idea. Nested Fenwick loops aggregate a rectangle through four prefixes.
Worked example. Update (1,2), then query a rectangle containing that cell.
Practice. Compare random rectangles against a matrix.

```java
package notebook;

/** Point additions and rectangle sums on a fixed zero-indexed grid. */
public final class Fenwick2D {
    private final int rows, columns;
    private final long[][] bit;

    public Fenwick2D(int rows, int columns) {
        if (rows < 0 || columns < 0) throw new IllegalArgumentException();
```

```
        this.rows = rows; this.columns = columns;
        bit = new long[rows + 1][columns + 1];
    }

    /** Add delta to one cell. O(log rows * log columns). */
    public void add(int row, int column, long delta) {
        if (row < 0 || row >= rows || column < 0 || column >= columns) throw new IndexOutOfBoundsException();
        for (int r = row + 1; r <= rows; r += r & -r)
            for (int c = column + 1; c <= columns; c += c & -c)
                bit[r][c] = Math.addExact(bit[r][c], delta);
    }

    /** Sum [0,row) x [0,column). O(log rows * log columns). */
    public long prefix(int row, int column) {
        if (row < 0 || row > rows || column < 0 || column > columns) throw new IndexOutOfBoundsException();
        long result = 0;
        for (int r = row; r > 0; r -= r & -r)
            for (int c = column; c > 0; c -= c & -c)
                result = Math.addExact(result, bit[r][c]);
        return result;
    }

    /** Sum [top,bottom) x [left,right). O(log rows * log columns). */
    public long rectangle(int top, int left, int bottom, int right) {
        if (top > bottom || left > right) throw new IndexOutOfBoundsException();
        return Math.subtractExact(Math.subtractExact(prefix(bottom, right), prefix(top, right)),
                Math.subtractExact(prefix(bottom, left), prefix(top, left)));
    }
}
```

3.5 Mo offline distinct queries

Level Intermediate
Use when Answer many static range distinct queries.
Cost Typical O((n+q) sqrt n).
Watch for All queries must be known in advance.

Core idea. Order queries by left block to limit pointer movement.
Worked example. For [1,2,1], distinct [0,3) is 2.
Practice. Compare with a HashSet on small arrays.

```
package notebook;

import java.util.Arrays;
import java.util.HashMap;
import java.util.Map;

/** Offline static range queries ordered by Mo's square-root decomposition. */
public final class MoQueries {
    private MoQueries() {}

    /** Distinct-value counts for half-open [left,right) queries. O((n+q)*sqrt(n)) typical. */
    public static int[] distinctCounts(int[] values, int[][] ranges) {
        int n = values.length, block = Math.max(1, (int) Math.sqrt(n));
        Map<Integer, Integer> compressed = new HashMap<>();
        int[] ids = new int[n];
        for (int i = 0; i < n; i++) {
            if (!compressed.containsKey(values[i])) compressed.put(values[i], compressed.size());
            ids[i] = compressed.get(values[i]);
        }
        Integer[] order = new Integer[ranges.length];
        for (int i = 0; i < ranges.length; i++) {
            if (ranges[i].length != 2 || ranges[i][0] < 0 || ranges[i][0] > ranges[i][1]
                    || ranges[i][1] > n) throw new IllegalArgumentException("invalid range");
            order[i] = i;
        }
        Arrays.sort(order, (a, b) -> {
            int leftBlock = ranges[a][0] / block, rightBlock = ranges[b][0] / block;
```

```
            if (leftBlock != rightBlock) return Integer.compare(leftBlock, rightBlock);
            return leftBlock % 2 == 0 ? Integer.compare(ranges[a][1], ranges[b][1])
                    : Integer.compare(ranges[b][1], ranges[a][1]);
        });
        int[] frequency = new int[compressed.size()], result = new int[ranges.length];
        int left = 0, right = 0, distinct = 0;
        for (int index : order) {
            int targetLeft = ranges[index][0], targetRight = ranges[index][1];
            while (left > targetLeft) if (frequency[ids[--left]]++ == 0) distinct++;
            while (right < targetRight) if (frequency[ids[right++]]++ == 0) distinct++;
            while (left < targetLeft) if (--frequency[ids[left++]] == 0) distinct--;
            while (right > targetRight) if (--frequency[ids[--right]] == 0) distinct--;
            result[index] = distinct;
        }
        return result;
    }
}
```

3.6 Li Chao line container

Level Advanced
Use when Query the minimum of lines at known coordinates.
Cost O(log domain) per operation.
Watch for Register every x and avoid long overflow.

Core idea. Keep the better line at the midpoint and send the other toward the side where it may win.
Worked example. For y=x and y=-x, the minimum at x=3 is -3.
Practice. Compare against direct line evaluation.

```
package notebook;

import java.util.Arrays;

/** Minimum of linear functions at a fixed set of query x-coordinates. */
public final class LiChaoTree {
    private record Line(long slope, long intercept) {
        long value(long x) { return Math.addExact(Math.multiplyExact(slope, x), intercept); }
    }
    private final long[] xs;
    private final Line[] lines;

    public LiChaoTree(long[] queryCoordinates) {
        if (queryCoordinates.length == 0) throw new IllegalArgumentException("empty domain");
        xs = queryCoordinates.clone();
        Arrays.sort(xs);
        for (int i = 1; i < xs.length; i++) if (xs[i] == xs[i - 1])
            throw new IllegalArgumentException("query coordinates must be distinct");
        lines = new Line[4 * xs.length];
    }

    /** Add y = slope*x + intercept. O(log domain size). */
    public void add(long slope, long intercept) { add(1, 0, xs.length, new Line(slope, intercept)); }

    private void add(int node, int left, int right, Line incoming) {
        if (lines[node] == null) { lines[node] = incoming; return; }
        int middle = (left + right) >>> 1;
        boolean betterLeft = incoming.value(xs[left]) < lines[node].value(xs[left]);
        boolean betterMiddle = incoming.value(xs[middle]) < lines[node].value(xs[middle]);
        if (betterMiddle) { Line old = lines[node]; lines[node] = incoming; incoming = old; }
        if (right - left == 1) return;
        if (betterLeft != betterMiddle) add(node * 2, left, middle, incoming);
        else add(node * 2 + 1, middle, right, incoming);
    }

    /** Minimum at a registered x; Long.MAX_VALUE when no line was added. O(log domain size). */
    public long query(long x) {
        int index = Arrays.binarySearch(xs, x);
        if (index < 0) throw new IllegalArgumentException("x was not registered");
```

```
        int node = 1, left = 0, right = xs.length;
        long result = Long.MAX_VALUE;
        while (true) {
            if (lines[node] != null) result = Math.min(result, lines[node].value(x));
            if (right - left == 1) return result;
            int middle = (left + right) >>> 1;
            if (index < middle) { node *= 2; right = middle; }
            else { node = node * 2 + 1; left = middle; }
        }
    }
}
```

4 Dynamic programming

State, transition, base case, and iteration order should be explicit.

4.1 First dynamic programs

Level Beginner
Use when Reuse answers to smaller states for sequences, coin change, or grids.
Cost O(n), O(amount * coin types), or O(rows * columns).
Watch for Path counts can overflow long; coin denominations must be positive.

Core idea. Define the state, base case, transition, and answer before writing loops. An iterative table avoids repeated recursion.
Worked example. For coins 1, 3, 4, amount 6 needs two coins: 3 + 3.
Practice. Write a table by hand for a small input and compare it with the program.

```
package notebook;

import java.util.Arrays;

/** First DP examples: identify a state, base case, transition, and answer. */
public final class BeginnerDynamicProgramming {
    private BeginnerDynamicProgramming() {}

    /** Fibonacci F(n), with F(0)=0 and F(1)=1. O(n) time; n must be at most 92. */
    public static long fibonacci(int n) {
        if (n < 0 || n > 92) throw new IllegalArgumentException("n must be in [0,92]");
        long previous = 0, current = 1;
        for (int i = 0; i < n; i++) { long next = previous + current; previous = current; current = next; }
        return previous;
    }

    /** Minimum coins for an amount, or -1 if impossible. O(amount * coins.length) time. */
    public static int minimumCoins(int[] coins, int amount) {
        if (amount < 0) throw new IllegalArgumentException("negative amount");
        int[] dp = new int[amount + 1];
        Arrays.fill(dp, amount + 1);
        dp[0] = 0;
        for (int value = 1; value <= amount; value++)
            for (int coin : coins) {
                if (coin <= 0) throw new IllegalArgumentException("coins must be positive");
                if (coin <= value) dp[value] = Math.min(dp[value], dp[value - coin] + 1);
            }
        return dp[amount] > amount ? -1 : dp[amount];
    }

    /** Number of paths in a grid, moving right or down around blocked cells. O(rows*columns). */
    public static long gridPaths(boolean[][] blocked) {
        int rows = blocked.length;
        if (rows == 0) return 0;
        int columns = blocked[0].length;
```

```
        for (boolean[] row : blocked) if (row.length != columns) throw new IllegalArgumentException("ragged grid");
        if (columns == 0) return 0;
        long[] dp = new long[columns];
        dp[0] = blocked[0][0] ? 0 : 1;
        for (int row = 0; row < rows; row++)
            for (int column = 0; column < columns; column++) {
                if (blocked[row][column]) dp[column] = 0;
                else if (column > 0) dp[column] = Math.addExact(dp[column], dp[column - 1]);
            }
        return dp[columns - 1];
    }
}
```

4.2 Classic DP patterns

Level Intermediate
Use when Solve knapsack, subsequence, edit, partition, and counting problems.
Cost Often O(n*capacity) or O(n^2).
Watch for Counts can exceed long.

Core idea. Many contest DPs differ only in the state and transition; order must respect dependencies.
Worked example. In 0/1 knapsack, iterate capacity downward so an item is used at most once.
Practice. Write a brute-force subset oracle for small n.

```
package notebook;

/** Small contest DP routines. Counts are long where practical and may overflow. */
public final class DynamicProgramming {
    private DynamicProgramming() {}

    /** Number of unordered combinations; positive coin values required. O(coins * target). */
    public static long coinChange(int[] coins, int target) {
        if (target < 0) return 0;
        long[] dp = new long[target + 1];
        dp[0] = 1;
        for (int coin : coins) {
            if (coin <= 0) throw new IllegalArgumentException("coin must be positive");
            for (int s = coin; s <= target; s++) dp[s] += dp[s - coin];
        }
        return dp[target];
    }

    /** Number of ordered ways to climb using steps of 1, 2, or 3. O(n). */
    public static long countWays(int n) {
        if (n < 0) return 0;
        long[] dp = new long[n + 1];
        dp[0] = 1;
        for (int i = 1; i <= n; i++)
            for (int step = 1; step <= 3; step++)
                if (i >= step) dp[i] += dp[i - step];
        return dp[n];
    }

    /** Levenshtein distance. O(a.length * b.length) time, O(b.length) space. */
    public static int editDistance(String a, String b) {
        int[] prev = new int[b.length() + 1];
        for (int j = 0; j <= b.length(); j++) prev[j] = j;
        for (int i = 1; i <= a.length(); i++) {
            int[] cur = new int[b.length() + 1];
            cur[0] = i;
            for (int j = 1; j <= b.length(); j++)
                cur[j] = a.charAt(i - 1) == b.charAt(j - 1) ? prev[j - 1]
                    : 1 + Math.min(prev[j - 1], Math.min(prev[j], cur[j - 1]));
            prev = cur;
        }
        return prev[b.length()];
    }
```

```java
    /** 0/1 knapsack with nonnegative weights and capacity. O(items * capacity). */
    public static long knapsack(int capacity, int[] weights, int[] values) {
        if (capacity < 0 || weights.length != values.length) throw new IllegalArgumentException();
        long[] dp = new long[capacity + 1];
        for (int i = 0; i < weights.length; i++) {
            if (weights[i] < 0) throw new IllegalArgumentException("negative weight");
            for (int w = capacity; w >= weights[i]; w--)
                dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
        }
        return dp[capacity];
    }

    /** Longest common subsequence length. O(mn) time, O(n) space. */
    public static int lcs(String a, String b) {
        int[] dp = new int[b.length() + 1];
        for (int i = 1; i <= a.length(); i++) {
            int diag = 0;
            for (int j = 1; j <= b.length(); j++) {
                int old = dp[j];
                dp[j] = a.charAt(i - 1) == b.charAt(j - 1) ? diag + 1
                    : Math.max(dp[j], dp[j - 1]);
                diag = old;
            }
        }
        return dp[b.length()];
    }

    /** Strictly increasing subsequence length. O(n log n). */
    public static int lis(int[] a) {
        int[] tails = new int[a.length];
        int size = 0;
        for (int x : a) {
            int lo = 0, hi = size;
            while (lo < hi) {
                int mid = (lo + hi) >>> 1;
                if (tails[mid] < x) lo = mid + 1;
                else hi = mid;
            }
            tails[lo] = x;
            if (lo == size) size++;
        }
        return size;
    }

    /** Maximum product after at least one cut; n >= 2. O(n). */
    public static long maxProductCutting(int n) {
        if (n < 2) throw new IllegalArgumentException();
        if (n <= 3) return n - 1;
        long result = 1;
        while (n > 4) { result = Math.multiplyExact(result, 3); n -= 3; }
        return Math.multiplyExact(result, n);
    }

    /** Minimum difference between subset sums; nonnegative values. O(n * sum) time. */
    public static int minimumPartition(int[] a) {
        int sum = 0;
        for (int x : a) {
            if (x < 0) throw new IllegalArgumentException("negative value");
            sum = Math.addExact(sum, x);
        }
        boolean[] dp = new boolean[sum / 2 + 1];
        dp[0] = true;
        for (int x : a)
            for (int s = dp.length - 1; s >= x; s--) dp[s] |= dp[s - x];
        for (int s = dp.length - 1; s >= 0; s--) if (dp[s]) return sum - 2 * s;
        throw new AssertionError();
    }

    /** Subset sum for nonnegative values and target. O(n * target). */
    public static boolean subsetSum(int[] a, int target) {
        if (target < 0) return false;
        boolean[] dp = new boolean[target + 1];
```

```
        dp[0] = true;
        for (int x : a) {
            if (x < 0) throw new IllegalArgumentException("negative value");
            for (int s = target; s >= x; s--) dp[s] |= dp[s - x];
        }
        return dp[target];
    }
}
```

4.3 Subset, interval, scheduling, digit DP

Level Advanced
Use when Use richer states than one-dimensional DP.
Cost O(n^2 2^n), O(n^3), or O(n log n).
Watch for Traveling salesperson limits n to 20.

Core idea. Bitmasks, intervals, and digits provide compact state descriptions for harder counting and optimization.
Worked example. A TSP mask records visited vertices, while last records the final vertex.
Practice. Estimate states and memory before allocating a table.

```
package notebook;

import java.util.Arrays;
import java.util.Comparator;

/** DP patterns beyond the standard one-dimensional and sequence examples. */
public final class AdvancedDynamicProgramming {
    private AdvancedDynamicProgramming() {}

    /** Minimum Hamiltonian cycle starting and ending at vertex 0. O(n^2 2^n); n<=20. */
    public static long travelingSalesperson(long[][] distance) {
        int n = distance.length;
        if (n == 0) return 0;
        if (n > 20) throw new IllegalArgumentException("n must be at most 20");
        for (long[] row : distance) if (row.length != n) throw new IllegalArgumentException("square matrix
     required");
        long inf = Long.MAX_VALUE / 4;
        long[][] dp = new long[1 << n][n];
        for (long[] row : dp) Arrays.fill(row, inf);
        dp[1][0] = 0;
        for (int mask = 1; mask < (1 << n); mask += 2)
            for (int last = 0; last < n; last++) {
                if (dp[mask][last] == inf) continue;
                for (int next = 0; next < n; next++) if ((mask & (1 << next)) == 0)
                    dp[mask | (1 << next)][next] = Math.min(
                        dp[mask | (1 << next)][next], dp[mask][last] + distance[last][next]);
            }
        long answer = inf;
        for (int last = 0; last < n; last++)
            answer = Math.min(answer, dp[(1 << n) - 1][last] + distance[last][0]);
        return answer;
    }

    /** Matrix-chain multiplication cost; dimensions[i-1] x dimensions[i]. O(n^3). */
    public static long matrixChainCost(int[] dimensions) {
        int n = dimensions.length - 1;
        if (n <= 0) return 0;
        long[][] dp = new long[n][n];
        for (int length = 2; length <= n; length++)
            for (int left = 0; left + length <= n; left++) {
                int right = left + length - 1;
                dp[left][right] = Long.MAX_VALUE;
                for (int split = left; split < right; split++) {
                    long product = (long) dimensions[left] * dimensions[split + 1] * dimensions[right + 1];
                    dp[left][right] = Math.min(dp[left][right], dp[left][split] + dp[split + 1][right] + product);
                }
            }
        return dp[0][n - 1];
```

```
    }

    /** Maximum profit from non-overlapping [start,end) jobs. O(n log n). */
    public static long weightedIntervals(long[][] jobs) {
        long[][] sorted = jobs.clone();
        for (int i = 0; i < sorted.length; i++) {
            if (sorted[i].length != 3 || sorted[i][0] > sorted[i][1]) throw new IllegalArgumentException();
            sorted[i] = sorted[i].clone();
        }
        Arrays.sort(sorted, Comparator.comparingLong(a -> a[1]));
        long[] ends = new long[sorted.length], dp = new long[sorted.length + 1];
        for (int i = 0; i < sorted.length; i++) ends[i] = sorted[i][1];
        for (int i = 0; i < sorted.length; i++) {
            int lo = 0, hi = i;
            while (lo < hi) {
                int mid = (lo + hi) >>> 1;
                if (ends[mid] <= sorted[i][0]) lo = mid + 1;
                else hi = mid;
            }
            dp[i + 1] = Math.max(dp[i], dp[lo] + sorted[i][2]);
        }
        return dp[sorted.length];
    }

    /** Count integers in [0, bound] without equal adjacent decimal digits. O(digits*10). */
    public static long countNoEqualAdjacent(long bound) {
        if (bound < 0) return 0;
        char[] digits = Long.toString(bound).toCharArray();
        long[][][] dp = new long[2][11][2];
        dp[1][10][0] = 1;
        for (char digit : digits) {
            long[][][] next = new long[2][11][2];
            for (int tight = 0; tight <= 1; tight++)
                for (int previous = 0; previous <= 10; previous++)
                    for (int started = 0; started <= 1; started++) {
                        long ways = dp[tight][previous][started];
                        for (int value = 0; value <= (tight == 1 ? digit - '0' : 9); value++) {
                            int nextTight = tight == 1 && value == digit - '0' ? 1 : 0;
                            int nextStarted = started == 1 || value != 0 ? 1 : 0;
                            if (nextStarted == 1 && started == 1 && previous == value) continue;
                            int nextPrevious = nextStarted == 1 ? value : 10;
                            next[nextTight][nextPrevious][nextStarted] += ways;
                        }
                    }
            dp = next;
        }
        long result = 0;
        for (long[][] state : dp) for (long[] prev : state) for (long ways : prev) result += ways;
        return result;
    }
}
```

4.4 Knuth optimized adjacent merging

Level Advanced
Use when Minimize adjacent merge cost for nonnegative weights.
Cost O(n^2) time and space.
Watch for Split monotonicity depends on this recurrence.

Core idea. Optimal split positions lie between those of neighboring intervals.
Worked example. Weights [1,2,3] have optimal cost 9.
Practice. Compare against cubic interval DP.

```
package notebook;

/** Optimal cost to merge adjacent weighted groups using Knuth's monotone split bounds. */
public final class KnuthOptimization {
    private KnuthOptimization() {}
```

```
    /** Nonnegative weights; each merge costs the sum of the merged interval. O(n^2) time and space. */
    public static long optimalAdjacentMergeCost(long[] weights) {
        int n = weights.length;
        for (long weight : weights) if (weight < 0) throw new IllegalArgumentException("weights must be
     nonnegative");
        if (n <= 1) return 0;
        long[] prefix = new long[n + 1];
        for (int i = 0; i < n; i++) {
            prefix[i + 1] = Math.addExact(prefix[i], weights[i]);
        }
        long[][] dp = new long[n][n];
        int[][] split = new int[n][n];
        for (int i = 0; i < n; i++) split[i][i] = i;
        for (int length = 2; length <= n; length++)
            for (int left = 0; left + length <= n; left++) {
                int right = left + length - 1;
                long best = Long.MAX_VALUE;
                int first = split[left][right - 1], last = Math.min(right - 1, split[left + 1][right]);
                for (int middle = first; middle <= last; middle++) {
                    long cost = Math.addExact(Math.addExact(dp[left][middle], dp[middle + 1][right]),
                            prefix[right + 1] - prefix[left]);
                    if (cost < best) { best = cost; split[left][right] = middle; }
                }
                dp[left][right] = best;
            }
        return dp[0][n - 1];
    }
}
```

5 Graphs

Distinguish directed and undirected edges before choosing an algorithm.

5.1 Adjacency lists, BFS, and grid flood fill

Level Beginner
Use when Explore an unweighted graph or count regions in a grid.
Cost O(V+E) for graphs; O(rows * columns) for grids.
Watch for For undirected graphs, insert both directions; BFS distance counts edges.

Core idea. An adjacency list stores each vertex's neighbors. BFS visits vertices in layers, so its first visit gives the shortest unweighted distance.
Worked example. For edges 0-1 and 1-2, BFS from 0 returns distances 0, 1, 2.
Practice. Draw a graph with an unreachable vertex and check the returned -1 distance.

```
package notebook;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** First graph routines using adjacency lists and an explicit queue. */
public final class BeginnerGraphs {
    private BeginnerGraphs() {}

    /** Build an undirected adjacency list from zero-indexed edges. O(V+E) time and space. */
    public static List<List<Integer>> undirectedGraph(int vertices, int[][] edges) {
        if (vertices < 0) throw new IllegalArgumentException("negative vertex count");
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < vertices; i++) graph.add(new ArrayList<>());
        for (int[] edge : edges) {
```

```
            if (edge.length != 2) throw new IllegalArgumentException("edge must contain two vertices");
            graph.get(edge[0]).add(edge[1]);
            graph.get(edge[1]).add(edge[0]);
        }
        return graph;
    }

    /** BFS distances from source; -1 means unreachable. O(V+E) time. */
    public static int[] distances(List<List<Integer>> graph, int source) {
        int[] distance = new int[graph.size()];
        Arrays.fill(distance, -1);
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        distance[source] = 0;
        queue.add(source);
        while (!queue.isEmpty()) {
            int vertex = queue.remove();
            for (int next : graph.get(vertex)) if (distance[next] == -1) {
                distance[next] = distance[vertex] + 1;
                queue.add(next);
            }
        }
        return distance;
    }

    /** Count four-neighbor components of true cells. O(rows*columns) time and space. */
    public static int gridComponents(boolean[][] open) {
        int rows = open.length;
        if (rows == 0) return 0;
        int columns = open[0].length;
        for (boolean[] row : open) if (row.length != columns) throw new IllegalArgumentException("ragged grid");
        boolean[][] seen = new boolean[rows][columns];
        int[] dr = {-1, 1, 0, 0}, dc = {0, 0, -1, 1};
        ArrayDeque<int[]> queue = new ArrayDeque<>();
        int components = 0;
        for (int row = 0; row < rows; row++) for (int column = 0; column < columns; column++) {
            if (!open[row][column] || seen[row][column]) continue;
            components++;
            seen[row][column] = true;
            queue.add(new int[] {row, column});
            while (!queue.isEmpty()) {
                int[] cell = queue.remove();
                for (int direction = 0; direction < 4; direction++) {
                    int nextRow = cell[0] + dr[direction], nextColumn = cell[1] + dc[direction];
                    if (nextRow < 0 || nextRow >= rows || nextColumn < 0 || nextColumn >= columns
                            || !open[nextRow][nextColumn] || seen[nextRow][nextColumn]) continue;
                    seen[nextRow][nextColumn] = true;
                    queue.add(new int[] {nextRow, nextColumn});
                }
            }
        }
        return components;
    }
}
```

5.2 Traversal, shortest paths, MST

Level Intermediate
Use when Use BFS, DFS, topological order, Dijkstra, or Kruskal.
Cost O(V+E) to O(E log V).
Watch for Dijkstra requires nonnegative weights.
Dependencies structures

Core idea. Represent edges explicitly, then select traversal or optimization from the weight rules.
Worked example. BFS finds unweighted distances; Dijkstra extends this to nonnegative weights.
Practice. Test disconnected vertices and parallel edges.

```
package notebook;
```

```java
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

/** Zero-indexed graph algorithms. */
public final class Graphs {
    private Graphs() {}

    /** Directed adjacency list. For undirected graphs add both directions. */
    public static List<List<Integer>> newGraph(int n) {
        if (n < 0) throw new IllegalArgumentException();
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        return graph;
    }

    /** Unweighted shortest-path distances; unreachable vertices have -1. O(V+E). */
    public static int[] bfs(List<List<Integer>> graph, int start) {
        int[] dist = new int[graph.size()];
        Arrays.fill(dist, -1);
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        dist[start] = 0; queue.add(start);
        while (!queue.isEmpty()) {
            int v = queue.remove();
            for (int to : graph.get(v)) if (dist[to] == -1) {
                dist[to] = dist[v] + 1; queue.add(to);
            }
        }
        return dist;
    }

    /** Iterative DFS visitation order. O(V+E). */
    public static List<Integer> dfs(List<List<Integer>> graph, int start) {
        boolean[] seen = new boolean[graph.size()];
        ArrayDeque<Integer> stack = new ArrayDeque<>();
        List<Integer> order = new ArrayList<>();
        stack.push(start);
        while (!stack.isEmpty()) {
            int v = stack.pop();
            if (seen[v]) continue;
            seen[v] = true; order.add(v);
            List<Integer> neighbors = graph.get(v);
            for (int i = neighbors.size() - 1; i >= 0; i--) if (!seen[neighbors.get(i)])
     stack.push(neighbors.get(i));
        }
        return order;
    }

    /** Kahn topological order; throws if the directed graph contains a cycle. */
    public static List<Integer> topologicalSort(List<List<Integer>> graph) {
        int[] indegree = new int[graph.size()];
        for (List<Integer> edges : graph) for (int to : edges) indegree[to]++;
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int i = 0; i < indegree.length; i++) if (indegree[i] == 0) queue.add(i);
        List<Integer> result = new ArrayList<>();
        while (!queue.isEmpty()) {
            int v = queue.remove(); result.add(v);
            for (int to : graph.get(v)) if (--indegree[to] == 0) queue.add(to);
        }
        if (result.size() != graph.size()) throw new IllegalArgumentException("graph has a cycle");
        return result;
    }

    public record WeightedEdge(int to, long weight) {}

    /** Dijkstra; requires nonnegative weights. Unreachable vertices have Long.MAX_VALUE. */
    public static long[] dijkstra(List<List<WeightedEdge>> graph, int start) {
        long[] dist = new long[graph.size()];
```

```
        Arrays.fill(dist, Long.MAX_VALUE);
        PriorityQueue<long[]> queue = new PriorityQueue<>(Comparator.comparingLong(a -> a[0]));
        dist[start] = 0; queue.add(new long[] {0, start});
        while (!queue.isEmpty()) {
            long[] current = queue.remove();
            long d = current[0]; int v = (int) current[1];
            if (d != dist[v]) continue;
            for (WeightedEdge edge : graph.get(v)) {
                if (edge.weight < 0) throw new IllegalArgumentException("negative edge");
                if (d > Long.MAX_VALUE - edge.weight) continue;
                long candidate = d + edge.weight;
                if (candidate < dist[edge.to]) {
                    dist[edge.to] = candidate;
                    queue.add(new long[] {candidate, edge.to});
                }
            }
        }
        return dist;
    }

    public record Edge(int from, int to, long weight) {}

    /** Kruskal MST; throws if disconnected. O(E log E). */
    public static long kruskal(int n, List<Edge> edges) {
        List<Edge> sorted = new ArrayList<>(edges);
        sorted.sort(Comparator.comparingLong(Edge::weight));
        DataStructures.DSU dsu = new DataStructures.DSU(n);
        long total = 0;
        for (Edge edge : sorted) if (dsu.union(edge.from, edge.to)) total = Math.addExact(total, edge.weight);
        if (n > 0 && dsu.components() != 1) throw new IllegalArgumentException("disconnected graph");
        return total;
    }
}
```

5.3 Components, bipartite, 0-1 BFS, Floyd, Euler

Level Intermediate
Use when Handle graph properties and special shortest paths.
Cost Usually O(V+E); Floyd is O(V^3).
Watch for Undirected edges require both directions.
Dependencies graphs

Core idea. Special graph structure can make a hard general problem simpler.
Worked example. A deque handles 0-weight edges at the front and 1-weight edges at the back.
Practice. Compare 0-1 BFS against Dijkstra on small graphs.

```
package notebook;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.List;

/** Graph routines that complement BFS, DFS, Dijkstra, and Kruskal. */
public final class GraphEssentials {
    private GraphEssentials() {}

    /** Component labels for an undirected adjacency list. O(V+E). */
    public static int[] components(List<List<Integer>> graph) {
        int[] label = new int[graph.size()];
        Arrays.fill(label, -1);
        int count = 0;
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int start = 0; start < graph.size(); start++) {
            if (label[start] != -1) continue;
            label[start] = count; queue.add(start);
```

```
            while (!queue.isEmpty()) {
                int v = queue.remove();
                for (int to : graph.get(v)) if (label[to] == -1) {
                    label[to] = count; queue.add(to);
                }
            }
            count++;
        }
        return label;
    }

    /** Two-color an undirected graph; return null if it is not bipartite. O(V+E). */
    public static int[] bipartiteColors(List<List<Integer>> graph) {
        int[] color = new int[graph.size()];
        Arrays.fill(color, -1);
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int start = 0; start < graph.size(); start++) {
            if (color[start] != -1) continue;
            color[start] = 0; queue.add(start);
            while (!queue.isEmpty()) {
                int v = queue.remove();
                for (int to : graph.get(v)) {
                    if (color[to] == color[v]) return null;
                    if (color[to] == -1) { color[to] = color[v] ^ 1; queue.add(to); }
                }
            }
        }
        return color;
    }

    public record BinaryEdge(int to, int weight) {}

    /** Shortest paths with edge weights exactly 0 or 1. O(V+E). */
    public static int[] zeroOneBfs(List<List<BinaryEdge>> graph, int source) {
        int[] dist = new int[graph.size()];
        Arrays.fill(dist, Integer.MAX_VALUE);
        ArrayDeque<Integer> deque = new ArrayDeque<>();
        dist[source] = 0; deque.add(source);
        while (!deque.isEmpty()) {
            int v = deque.removeFirst();
            for (BinaryEdge edge : graph.get(v)) {
                if (edge.weight != 0 && edge.weight != 1) throw new IllegalArgumentException("weight must be 0 or
     1");
                int candidate = dist[v] + edge.weight;
                if (candidate < dist[edge.to]) {
                    dist[edge.to] = candidate;
                    if (edge.weight == 0) deque.addFirst(edge.to);
                    else deque.addLast(edge.to);
                }
            }
        }
        return dist;
    }

    /** All-pairs shortest paths; Long.MAX_VALUE means unreachable. O(V^3). */
    public static long[][] floydWarshall(long[][] weights) {
        int n = weights.length;
        long[][] dist = new long[n][n];
        for (int i = 0; i < n; i++) {
            if (weights[i].length != n) throw new IllegalArgumentException("matrix must be square");
            dist[i] = weights[i].clone();
            dist[i][i] = Math.min(dist[i][i], 0);
        }
        for (int k = 0; k < n; k++)
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++) {
                    if (dist[i][k] == Long.MAX_VALUE || dist[k][j] == Long.MAX_VALUE) continue;
                    long candidate = Math.addExact(dist[i][k], dist[k][j]);
                    if (candidate < dist[i][j]) dist[i][j] = candidate;
                }
        for (int i = 0; i < n; i++) if (dist[i][i] < 0) throw new IllegalArgumentException("negative cycle");
        return dist;
```

```
    }

    /** Eulerian trail in a directed multigraph, or empty list if none exists. O(V+E). */
    public static List<Integer> directedEulerTrail(int n, int[][] edges) {
        if (n == 0) return List.of();
        List<List<Integer>> graph = Graphs.newGraph(n);
        int[] in = new int[n], out = new int[n], next = new int[n];
        for (int[] edge : edges) {
            graph.get(edge[0]).add(edge[1]); out[edge[0]]++; in[edge[1]]++;
        }
        int start = -1, end = -1;
        for (int v = 0; v < n; v++) {
            int delta = out[v] - in[v];
            if (delta == 1) { if (start != -1) return List.of(); start = v; }
            else if (delta == -1) { if (end != -1) return List.of(); end = v; }
            else if (delta != 0) return List.of();
        }
        if ((start == -1) != (end == -1)) return List.of();
        if (start == -1) for (int v = 0; v < n; v++) if (out[v] > 0) { start = v; break; }
        if (start == -1) return List.of(0);
        ArrayDeque<Integer> stack = new ArrayDeque<>();
        List<Integer> trail = new ArrayList<>();
        stack.push(start);
        while (!stack.isEmpty()) {
            int v = stack.peek();
            if (next[v] < graph.get(v).size()) stack.push(graph.get(v).get(next[v]++));
            else trail.add(stack.pop());
        }
        if (trail.size() != edges.length + 1) return List.of();
        Collections.reverse(trail);
        return trail;
    }
}
```

5.4 SCC, Bellman-Ford, LCA, max flow

Level Advanced
Use when Analyze directed components, negative edges, ancestors, or flow.
Cost O(V+E) to O(VE), depending on method.
Watch for Bellman-Ford rejects reachable negative cycles.
Dependencies graphs

Core idea. SCCs compress directed cycles; flow and ancestor structures answer different global questions.
Worked example. Condensing each SCC produces a directed acyclic graph.
Practice. Draw a graph with two SCCs and one edge between them.

```
package notebook;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** Additional graph routines. Vertices are zero-indexed. */
public final class AdvancedGraphs {
    private AdvancedGraphs() {}

    /** Kosaraju SCC labels for a directed graph. O(V+E), iterative DFS. */
    public static int[] stronglyConnectedComponents(List<List<Integer>> graph) {
        int n = graph.size();
        List<List<Integer>> reverse = Graphs.newGraph(n);
        for (int v = 0; v < n; v++) for (int to : graph.get(v)) reverse.get(to).add(v);
        boolean[] seen = new boolean[n];
        List<Integer> order = new ArrayList<>();
        for (int start = 0; start < n; start++) {
            if (seen[start]) continue;
            ArrayDeque<int[]> stack = new ArrayDeque<>();
```

```
            seen[start] = true; stack.push(new int[] {start, 0});
            while (!stack.isEmpty()) {
                int[] frame = stack.peek();
                List<Integer> edges = graph.get(frame[0]);
                if (frame[1] == edges.size()) { order.add(frame[0]); stack.pop(); continue; }
                int to = edges.get(frame[1]++);
                if (!seen[to]) { seen[to] = true; stack.push(new int[] {to, 0}); }
            }
        }
        int[] component = new int[n];
        Arrays.fill(component, -1);
        int count = 0;
        for (int i = order.size() - 1; i >= 0; i--) {
            int start = order.get(i);
            if (component[start] != -1) continue;
            ArrayDeque<Integer> stack = new ArrayDeque<>();
            stack.push(start); component[start] = count;
            while (!stack.isEmpty()) {
                for (int to : reverse.get(stack.pop())) if (component[to] == -1) {
                    component[to] = count; stack.push(to);
                }
            }
            count++;
        }
        return component;
    }

    /** Bellman-Ford; throws if a reachable negative cycle exists. O(VE). */
    public static long[] bellmanFord(int n, List<Graphs.Edge> edges, int source) {
        long[] dist = new long[n];
        Arrays.fill(dist, Long.MAX_VALUE);
        dist[source] = 0;
        for (int pass = 0; pass < n; pass++) {
            boolean changed = false;
            for (Graphs.Edge edge : edges) {
                if (dist[edge.from()] == Long.MAX_VALUE) continue;
                long candidate = Math.addExact(dist[edge.from()], edge.weight());
                if (candidate < dist[edge.to()]) {
                    if (pass == n - 1) throw new IllegalArgumentException("reachable negative cycle");
                    dist[edge.to()] = candidate;
                    changed = true;
                }
            }
            if (!changed) break;
        }
        return dist;
    }

    /** Binary-lifting LCA for a connected, undirected tree. O(n log n) build, O(log n) query. */
    public static final class LCA {
        private final int[][] up;
        private final int[] depth;

        public LCA(List<List<Integer>> tree, int root) {
            int n = tree.size(), levels = 1;
            while ((1L << levels) <= Math.max(1, n)) levels++;
            up = new int[levels][n]; depth = new int[n];
            Arrays.fill(depth, -1);
            ArrayDeque<Integer> queue = new ArrayDeque<>();
            depth[root] = 0; up[0][root] = root; queue.add(root);
            while (!queue.isEmpty()) {
                int v = queue.remove();
                for (int to : tree.get(v)) if (depth[to] == -1) {
                    depth[to] = depth[v] + 1; up[0][to] = v; queue.add(to);
                }
            }
            for (int d : depth) if (d < 0) throw new IllegalArgumentException("disconnected tree");
            for (int k = 1; k < levels; k++)
                for (int v = 0; v < n; v++) up[k][v] = up[k - 1][up[k - 1][v]];
        }

        public int query(int a, int b) {
```

```
            if (depth[a] < depth[b]) { int t = a; a = b; b = t; }
            int delta = depth[a] - depth[b];
            for (int k = 0; delta > 0; k++, delta >>= 1) if ((delta & 1) != 0) a = up[k][a];
            if (a == b) return a;
            for (int k = up.length - 1; k >= 0; k--) if (up[k][a] != up[k][b]) {
                a = up[k][a]; b = up[k][b];
            }
            return up[0][a];
        }
    }

    /** Dinic maximum flow with nonnegative long capacities. */
    public static final class Dinic {
        private static final class Arc {
            int to, reverse;
            long capacity;
            Arc(int to, int reverse, long capacity) {
                this.to = to; this.reverse = reverse; this.capacity = capacity;
            }
        }
        private final List<List<Arc>> graph = new ArrayList<>();
        private final int[] level, next;
        public Dinic(int n) {
            if (n < 0) throw new IllegalArgumentException();
            for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
            level = new int[n]; next = new int[n];
        }
        public void addEdge(int from, int to, long capacity) {
            if (capacity < 0) throw new IllegalArgumentException();
            int forwardReverse = graph.get(to).size() + (from == to ? 1 : 0);
            Arc forward = new Arc(to, forwardReverse, capacity);
            Arc backward = new Arc(from, graph.get(from).size(), 0);
            graph.get(from).add(forward); graph.get(to).add(backward);
        }
        private boolean levels(int source, int sink) {
            Arrays.fill(level, -1);
            ArrayDeque<Integer> queue = new ArrayDeque<>();
            queue.add(source); level[source] = 0;
            while (!queue.isEmpty()) {
                int v = queue.remove();
                for (Arc arc : graph.get(v)) if (arc.capacity > 0 && level[arc.to] == -1) {
                    level[arc.to] = level[v] + 1; queue.add(arc.to);
                }
            }
            return level[sink] >= 0;
        }
        private long send(int v, int sink, long flow) {
            if (v == sink) return flow;
            List<Arc> arcs = graph.get(v);
            for (; next[v] < arcs.size(); next[v]++) {
                Arc arc = arcs.get(next[v]);
                if (arc.capacity == 0 || level[arc.to] != level[v] + 1) continue;
                long pushed = send(arc.to, sink, Math.min(flow, arc.capacity));
                if (pushed == 0) continue;
                arc.capacity -= pushed;
                Arc back = graph.get(arc.to).get(arc.reverse);
                back.capacity = Math.addExact(back.capacity, pushed);
                return pushed;
            }
            return 0;
        }
        public long maxFlow(int source, int sink) {
            if (source == sink) return 0;
            long total = 0;
            while (levels(source, sink)) {
                Arrays.fill(next, 0);
                long pushed;
                while ((pushed = send(source, sink, Long.MAX_VALUE)) > 0) total = Math.addExact(total, pushed);
            }
            return total;
        }
    }
```

```
}
```

5.5 Bridges and articulation points

Level Advanced
Use when Find critical edges or vertices in undirected graphs.
Cost O(V+E).
Watch for Recursive DFS may exhaust the Java stack.

Core idea. DFS discovery and low-link times reveal edges and vertices whose removal disconnects a graph.
Worked example. A tree edge is a bridge when the child cannot reach an ancestor of its parent.
Practice. Remove each edge in a tiny graph and compare component counts.

```
package notebook;

import java.util.ArrayList;
import java.util.List;

/** Bridges and articulation points in an undirected multigraph. Recursive DFS uses O(V) stack. */
public final class UndirectedCuts {
    private UndirectedCuts() {}

    public record Edge(int u, int v) {}
    public record Result(List<Integer> bridges, List<Integer> articulationPoints) {}

    /** Bridges are returned as indices into edges. O(V+E). */
    public static Result find(int n, List<Edge> edges) {
        List<List<int[]>> graph = new ArrayList<>();
        for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
        for (int i = 0; i < edges.size(); i++) {
            Edge edge = edges.get(i);
            graph.get(edge.u).add(new int[] {edge.v, i});
            graph.get(edge.v).add(new int[] {edge.u, i});
        }
        int[] tin = new int[n], low = new int[n];
        boolean[] cut = new boolean[n];
        List<Integer> bridges = new ArrayList<>();
        int[] timer = {1};
        for (int v = 0; v < n; v++) if (tin[v] == 0)
            dfs(v, -1, graph, tin, low, cut, bridges, timer);
        List<Integer> cuts = new ArrayList<>();
        for (int i = 0; i < n; i++) if (cut[i]) cuts.add(i);
        bridges.sort(null);
        return new Result(bridges, cuts);
    }

    private static void dfs(int v, int parentEdge, List<List<int[]>> graph, int[] tin, int[] low,
                            boolean[] cut, List<Integer> bridges, int[] timer) {
        tin[v] = low[v] = timer[0]++;
        int children = 0;
        for (int[] edge : graph.get(v)) {
            int to = edge[0], id = edge[1];
            if (id == parentEdge) continue;
            if (tin[to] != 0) low[v] = Math.min(low[v], tin[to]);
            else {
                dfs(to, id, graph, tin, low, cut, bridges, timer);
                low[v] = Math.min(low[v], low[to]);
                if (low[to] > tin[v]) bridges.add(id);
                if (parentEdge != -1 && low[to] >= tin[v]) cut[v] = true;
                children++;
            }
        }
        if (parentEdge == -1 && children > 1) cut[v] = true;
    }
}
```

5.6 2-SAT

Level Advanced
Use when Solve Boolean formulas with two-literal clauses.
Cost O(variables + clauses).
Watch for The API uses variable indices and Boolean values.
Dependencies advanced-graphs

Core idea. Translate each clause into implications, then test whether a variable and its negation share an SCC.
Worked example. The clause a OR b gives implications not-a to b and not-b to a.
Practice. Enumerate all assignments for formulas with at most ten variables.

```
package notebook;

import java.util.List;

/** Boolean 2-SAT via strongly connected components. O(variables + clauses). */
public final class TwoSat {
    private final int variables;
    private final List<List<Integer>> implication;

    public TwoSat(int variables) {
        if (variables < 0) throw new IllegalArgumentException();
        this.variables = variables;
        implication = Graphs.newGraph(2 * variables);
    }

    private int literal(int variable, boolean value) {
        if (variable < 0 || variable >= variables) throw new IndexOutOfBoundsException();
        return 2 * variable + (value ? 1 : 0);
    }

    /** Add (a == aValue) OR (b == bValue). */
    public void addOr(int a, boolean aValue, int b, boolean bValue) {
        int x = literal(a, aValue), y = literal(b, bValue);
        implication.get(x ^ 1).add(y);
        implication.get(y ^ 1).add(x);
    }

    /** Return one satisfying assignment, or null if unsatisfiable. */
    public boolean[] solve() {
        int[] component = AdvancedGraphs.stronglyConnectedComponents(implication);
        boolean[] assignment = new boolean[variables];
        for (int i = 0; i < variables; i++) {
            if (component[2 * i] == component[2 * i + 1]) return null;
            assignment[i] = component[2 * i + 1] > component[2 * i];
        }
        return assignment;
    }
}
```

5.7 Minimum-cost flow

Level Advanced
Use when Optimize flow cost under capacity limits.
Cost Depends on the number of augmentations.
Watch for Initial edge costs must be nonnegative.

Core idea. Residual reverse edges allow a later path to undo an earlier choice.
Worked example. After augmenting, add reverse capacity with the negated edge cost.
Practice. Check a case where rerouting improves the total cost.

```
package notebook;

import java.util.ArrayList;
```

```java
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;
import java.util.PriorityQueue;

/** Successive shortest augmenting paths with potentials; initial edge costs must be nonnegative. */
public final class MinCostFlow {
    public record Result(long flow, long cost) {}
    private static final class Edge {
        int to, reverse;
        long capacity, cost;
        Edge(int to, int reverse, long capacity, long cost) {
            this.to = to; this.reverse = reverse; this.capacity = capacity; this.cost = cost;
        }
    }
    private final List<List<Edge>> graph = new ArrayList<>();

    public MinCostFlow(int vertices) {
        if (vertices < 0) throw new IllegalArgumentException();
        for (int i = 0; i < vertices; i++) graph.add(new ArrayList<>());
    }

    public void addEdge(int from, int to, long capacity, long cost) {
        if (capacity < 0 || cost < 0) throw new IllegalArgumentException();
        int reverseIndex = graph.get(to).size() + (from == to ? 1 : 0);
        Edge forward = new Edge(to, reverseIndex, capacity, cost);
        Edge backward = new Edge(from, graph.get(from).size(), 0, -cost);
        graph.get(from).add(forward); graph.get(to).add(backward);
    }

    /** Send up to requestedFlow units. O(flow * E log V) for unit capacities. */
    public Result send(int source, int sink, long requestedFlow) {
        if (requestedFlow < 0) throw new IllegalArgumentException();
        int n = graph.size();
        long[] potential = new long[n];
        long flow = 0, cost = 0;
        while (flow < requestedFlow) {
            long[] dist = new long[n];
            int[] previousVertex = new int[n], previousEdge = new int[n];
            Arrays.fill(dist, Long.MAX_VALUE);
            PriorityQueue<long[]> queue = new PriorityQueue<>(Comparator.comparingLong(a -> a[0]));
            dist[source] = 0; queue.add(new long[] {0, source});
            while (!queue.isEmpty()) {
                long[] current = queue.remove();
                int v = (int) current[1];
                if (current[0] != dist[v]) continue;
                for (int i = 0; i < graph.get(v).size(); i++) {
                    Edge edge = graph.get(v).get(i);
                    if (edge.capacity == 0) continue;
                    long reduced = Math.addExact(edge.cost, potential[v] - potential[edge.to]);
                    long candidate = Math.addExact(dist[v], reduced);
                    if (candidate < dist[edge.to]) {
                        dist[edge.to] = candidate;
                        previousVertex[edge.to] = v;
                        previousEdge[edge.to] = i;
                        queue.add(new long[] {candidate, edge.to});
                    }
                }
            }
            if (dist[sink] == Long.MAX_VALUE) break;
            for (int v = 0; v < n; v++) if (dist[v] != Long.MAX_VALUE)
                potential[v] = Math.addExact(potential[v], dist[v]);
            long amount = requestedFlow - flow;
            for (int v = sink; v != source; v = previousVertex[v])
                amount = Math.min(amount, graph.get(previousVertex[v]).get(previousEdge[v]).capacity);
            for (int v = sink; v != source; v = previousVertex[v]) {
                Edge edge = graph.get(previousVertex[v]).get(previousEdge[v]);
                edge.capacity -= amount;
                Edge reverse = graph.get(v).get(edge.reverse);
                reverse.capacity += amount;
                cost = Math.addExact(cost, Math.multiplyExact(amount, edge.cost));
            }
```

```
            flow += amount;
        }
        return new Result(flow, cost);
    }
}
```

5.8 Bipartite matching and vertex cover

Level Intermediate
Use when Assign left vertices to right vertices.
Cost O(E sqrt V) matching.
Watch for The DFS phase is recursive.

Core idea. Layered augmenting paths build a maximum matching; alternating reachability gives a minimum cover.
Worked example. Edges 0-a, 1-a, 1-b allow a matching of size two.
Practice. Brute-force tiny matchings and validate the cover.

```
package notebook;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** Hopcroft-Karp matching and Konig minimum vertex cover in bipartite graphs. */
public final class BipartiteMatching {
    public record Cover(int[] left, int[] right) {}
    private final List<List<Integer>> edges;
    private final int[] leftMatch, rightMatch, distance;
    private int endDistance;

    public BipartiteMatching(int leftSize, int rightSize) {
        if (leftSize < 0 || rightSize < 0) throw new IllegalArgumentException();
        edges = new ArrayList<>();
        for (int i = 0; i < leftSize; i++) edges.add(new ArrayList<>());
        leftMatch = new int[leftSize]; rightMatch = new int[rightSize]; distance = new int[leftSize];
        Arrays.fill(leftMatch, -1); Arrays.fill(rightMatch, -1);
    }

    public void addEdge(int left, int right) {
        if (right < 0 || right >= rightMatch.length) throw new IndexOutOfBoundsException();
        edges.get(left).add(right);
    }

    /** Maximum matching size. O(E*sqrt(V)); DFS is recursive. */
    public int maximumMatching() {
        Arrays.fill(leftMatch, -1); Arrays.fill(rightMatch, -1);
        int size = 0;
        while (layers())
            for (int left = 0; left < leftMatch.length; left++)
                if (leftMatch[left] == -1 && augment(left)) size++;
        return size;
    }

    private boolean layers() {
        Arrays.fill(distance, -1);
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int left = 0; left < leftMatch.length; left++) if (leftMatch[left] == -1) {
            distance[left] = 0; queue.add(left);
        }
        endDistance = Integer.MAX_VALUE;
        while (!queue.isEmpty()) {
            int left = queue.remove();
            if (distance[left] >= endDistance) continue;
            for (int right : edges.get(left)) {
                int partner = rightMatch[right];
                if (partner == -1) endDistance = distance[left] + 1;
                else if (distance[partner] == -1) {
```

```
                    distance[partner] = distance[left] + 1;
                    queue.add(partner);
                }
            }
        }
        return endDistance != Integer.MAX_VALUE;
    }

    private boolean augment(int left) {
        for (int right : edges.get(left)) {
            int partner = rightMatch[right];
            if (partner == -1 && distance[left] + 1 == endDistance
                    || partner != -1 && distance[partner] == distance[left] + 1 && augment(partner)) {
                leftMatch[left] = right; rightMatch[right] = left; return true;
            }
        }
        distance[left] = -1;
        return false;
    }

    /** Minimum vertex cover; calls maximumMatching first. O(E*sqrt(V)). */
    public Cover minimumVertexCover() {
        maximumMatching();
        boolean[] seenLeft = new boolean[leftMatch.length], seenRight = new boolean[rightMatch.length];
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int left = 0; left < leftMatch.length; left++) if (leftMatch[left] == -1) {
            seenLeft[left] = true; queue.add(left);
        }
        while (!queue.isEmpty()) {
            int left = queue.remove();
            for (int right : edges.get(left)) {
                if (leftMatch[left] == right || seenRight[right]) continue;
                seenRight[right] = true;
                int partner = rightMatch[right];
                if (partner != -1 && !seenLeft[partner]) { seenLeft[partner] = true; queue.add(partner); }
            }
        }
        int leftCount = 0, rightCount = 0;
        for (boolean seen : seenLeft) if (!seen) leftCount++;
        for (boolean seen : seenRight) if (seen) rightCount++;
        int[] coverLeft = new int[leftCount], coverRight = new int[rightCount];
        leftCount = rightCount = 0;
        for (int i = 0; i < seenLeft.length; i++) if (!seenLeft[i]) coverLeft[leftCount++] = i;
        for (int i = 0; i < seenRight.length; i++) if (seenRight[i]) coverRight[rightCount++] = i;
        return new Cover(coverLeft, coverRight);
    }

    public int[] leftMatches() { return leftMatch.clone(); }
}
```

5.9 Global minimum cut

Level Advanced
Use when Find the lightest undirected weighted cut.
Cost O(V^3) time.
Watch for Symmetric nonnegative matrix; sums fit long.

Core idea. Stoer-Wagner grows a tight set and contracts its last two vertices.
Worked example. Two vertices joined by weight 7 have cut weight 7.
Practice. Enumerate all vertex subsets on tiny graphs.

```
package notebook;

/** Stoer-Wagner global minimum cut in a nonnegative undirected weighted graph. */
public final class GlobalMinCut {
    private GlobalMinCut() {}

    /** Symmetric adjacency matrix, zero diagonal. O(n^3) time, O(n^2) space. */
```

```
    public static long weight(long[][] weights) {
        int n = weights.length;
        if (n < 2) throw new IllegalArgumentException("at least two vertices required");
        long[][] graph = new long[n][n];
        for (long[] row : weights) if (row.length != n) throw new IllegalArgumentException("invalid matrix");
        for (int i = 0; i < n; i++) {
            if (weights[i][i] != 0) throw new IllegalArgumentException("invalid matrix");
            for (int j = 0; j < n; j++) {
                if (weights[i][j] < 0 || weights[i][j] != weights[j][i])
                    throw new IllegalArgumentException("matrix must be symmetric and nonnegative");
            }
            graph[i] = weights[i].clone();
        }
        int[] vertices = new int[n];
        for (int i = 0; i < n; i++) vertices[i] = i;
        long best = Long.MAX_VALUE;
        for (int remaining = n; remaining > 1; remaining--) {
            long[] connection = new long[n];
            boolean[] used = new boolean[n];
            int previous = -1;
            for (int step = 0; step < remaining; step++) {
                int selected = -1;
                for (int i = 0; i < remaining; i++) {
                    int vertex = vertices[i];
                    if (!used[vertex] && (selected == -1 || connection[vertex] > connection[selected]))
                        selected = vertex;
                }
                used[selected] = true;
                if (step == remaining - 1) {
                    best = Math.min(best, connection[selected]);
                    for (int i = 0; i < remaining; i++) {
                        int vertex = vertices[i];
                        if (vertex == selected || vertex == previous) continue;
                        graph[previous][vertex] = Math.addExact(graph[previous][vertex], graph[selected][vertex]);
                        graph[vertex][previous] = graph[previous][vertex];
                    }
                    for (int i = 0; i < remaining; i++) if (vertices[i] == selected) {
                        vertices[i] = vertices[remaining - 1]; break;
                    }
                } else {
                    previous = selected;
                    for (int i = 0; i < remaining; i++) {
                        int vertex = vertices[i];
                        if (!used[vertex]) connection[vertex] = Math.addExact(connection[vertex],
    graph[selected][vertex]);
                    }
                }
            }
        }
        return best;
    }
}
```

5.10 Exact spanning-tree count

Level Advanced
Use when Count spanning trees of an undirected multigraph.
Cost O(V^3) BigInteger arithmetic.
Watch for Dense exact arithmetic grows quickly; self-loops do not contribute.

Core idea. Kirchhoff removes one row and column of the graph Laplacian and takes the determinant.
Worked example. A triangle has three spanning trees.
Practice. Enumerate edge subsets for small multigraphs.

```
package notebook;

import java.math.BigInteger;
```

```java
/** Exact number of spanning trees in an undirected multigraph. */
public final class MatrixTreeTheorem {
    private MatrixTreeTheorem() {}

    /** Kirchhoff cofactor with a fraction-free Bareiss determinant. O(n^3) BigInteger work. */
    public static BigInteger count(int vertices, int[][] edges) {
        if (vertices < 0) throw new IllegalArgumentException();
        if (vertices == 0) return BigInteger.ZERO;
        if (vertices == 1) return BigInteger.ONE;
        BigInteger[][] laplacian = new BigInteger[vertices][vertices];
        for (int row = 0; row < vertices; row++)
            for (int column = 0; column < vertices; column++) laplacian[row][column] = BigInteger.ZERO;
        for (int[] edge : edges) {
            if (edge.length != 2 || edge[0] < 0 || edge[0] >= vertices || edge[1] < 0 || edge[1] >= vertices)
                throw new IllegalArgumentException("invalid edge");
            int a = edge[0], b = edge[1];
            if (a == b) continue;
            laplacian[a][a] = laplacian[a][a].add(BigInteger.ONE);
            laplacian[b][b] = laplacian[b][b].add(BigInteger.ONE);
            laplacian[a][b] = laplacian[a][b].subtract(BigInteger.ONE);
            laplacian[b][a] = laplacian[b][a].subtract(BigInteger.ONE);
        }
        int n = vertices - 1;
        BigInteger[][] matrix = new BigInteger[n][n];
        for (int row = 0; row < n; row++)
            System.arraycopy(laplacian[row], 0, matrix[row], 0, n);
        BigInteger previous = BigInteger.ONE;
        int sign = 1;
        for (int pivot = 0; pivot < n - 1; pivot++) {
            if (matrix[pivot][pivot].signum() == 0) {
                int replacement = pivot + 1;
                while (replacement < n && matrix[replacement][pivot].signum() == 0) replacement++;
                if (replacement == n) return BigInteger.ZERO;
                BigInteger[] swap = matrix[pivot]; matrix[pivot] = matrix[replacement]; matrix[replacement] = swap;
                sign = -sign;
            }
            BigInteger diagonal = matrix[pivot][pivot];
            for (int row = pivot + 1; row < n; row++)
                for (int column = pivot + 1; column < n; column++)
                    matrix[row][column] = matrix[row][column].multiply(diagonal)
                            .subtract(matrix[row][pivot].multiply(matrix[pivot][column])).divide(previous);
            for (int row = pivot + 1; row < n; row++) matrix[row][pivot] = BigInteger.ZERO;
            previous = diagonal;
        }
        BigInteger determinant = matrix[n - 1][n - 1];
        return sign < 0 ? determinant.negate() : determinant;
    }
}
```

5.11 Minimum-cost assignment

Level Intermediate
Use when Assign every worker to a distinct task at minimum total cost.
Cost O(rows^2 * columns) for rows <= columns.
Watch for Dense matrix model; arithmetic overflow is reported.

Core idea. Dual potentials maintain nonnegative reduced costs while augmenting the assignment.
Worked example. For [[4,1,3],[2,0,5]], the minimum assignment cost is 3.
Practice. Compare all column permutations for matrices with at most seven rows.

```java
package notebook;

import java.util.Arrays;

/** Minimum-cost assignment of every row to a distinct column. */
public final class HungarianAlgorithm {
    private HungarianAlgorithm() {}
    public record Assignment(long cost, int[] columnForRow) {}
```

```
    /** Rectangular dense assignment, rows <= columns. O(rows^2 * columns). */
    public static Assignment minimize(long[][] costs) {
        int rows = costs.length;
        if (rows == 0) return new Assignment(0, new int[0]);
        int columns = costs[0].length;
        if (columns < rows) throw new IllegalArgumentException("rows must not exceed columns");
        for (long[] row : costs) if (row.length != columns) throw new IllegalArgumentException("ragged cost matrix");
        long[] rowPotential = new long[rows + 1], columnPotential = new long[columns + 1];
        int[] matchedRow = new int[columns + 1], previousColumn = new int[columns + 1];
        for (int row = 1; row <= rows; row++) {
            matchedRow[0] = row;
            long[] minimum = new long[columns + 1];
            Arrays.fill(minimum, Long.MAX_VALUE);
            boolean[] used = new boolean[columns + 1];
            int currentColumn = 0;
            do {
                used[currentColumn] = true;
                int currentRow = matchedRow[currentColumn], nextColumn = -1;
                long delta = Long.MAX_VALUE;
                for (int column = 1; column <= columns; column++) if (!used[column]) {
                    long reduced = Math.subtractExact(Math.subtractExact(costs[currentRow - 1][column - 1],
                            rowPotential[currentRow]), columnPotential[column]);
                    if (reduced < minimum[column]) {
                        minimum[column] = reduced;
                        previousColumn[column] = currentColumn;
                    }
                    if (minimum[column] < delta) { delta = minimum[column]; nextColumn = column; }
                }
                if (nextColumn < 0 || delta == Long.MAX_VALUE) throw new IllegalArgumentException("no assignment");
                for (int column = 0; column <= columns; column++) {
                    if (used[column]) {
                        rowPotential[matchedRow[column]] = Math.addExact(rowPotential[matchedRow[column]], delta);
                        columnPotential[column] = Math.subtractExact(columnPotential[column], delta);
                    } else if (minimum[column] != Long.MAX_VALUE)
                        minimum[column] = Math.subtractExact(minimum[column], delta);
                }
                currentColumn = nextColumn;
            } while (matchedRow[currentColumn] != 0);
            do {
                int previous = previousColumn[currentColumn];
                matchedRow[currentColumn] = matchedRow[previous];
                currentColumn = previous;
            } while (currentColumn != 0);
        }
        int[] assignment = new int[rows];
        for (int column = 1; column <= columns; column++)
            if (matchedRow[column] != 0) assignment[matchedRow[column] - 1] = column - 1;
        long total = 0;
        for (int row = 0; row < rows; row++) total = Math.addExact(total, costs[row][assignment[row]]);
        return new Assignment(total, assignment);
    }
}
```

5.12 Binary submodular energy minimization

Level Advanced
Use when Minimize unary label costs plus pairwise submodular penalties.
Cost Dinic max-flow; runtime depends on graph shape.
Watch for Every pair must satisfy E00+E11 <= E01+E10; long arithmetic must fit.

Core idea. Rewrite each pair table as unary terms and a nonnegative directed cut edge.
Worked example. A pairwise disagreement penalty encourages adjacent labels to agree.
Practice. Enumerate all 2^n labelings on small instances and compare energy.

```
package notebook;

import java.util.ArrayDeque;
```

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** Exact minimum of binary unary and submodular pairwise energies by an s-t cut. */
public final class BinaryGraphCut {
    private BinaryGraphCut() {}
    public record Result(long energy, boolean[] labelOne) {}

    private static final class Edge {
        int to, reverse;
        long capacity;
        Edge(int to,int reverse,long capacity) { this.to=to; this.reverse=reverse; this.capacity=capacity; }
    }

    /**
     * Minimize sum_i unary[i][label_i] plus pairwise[e][label_u][label_v].
     * Each pair must satisfy E00+E11 <= E01+E10. Uses Dinic max flow.
     */
    public static Result minimize(long[][] unary, int[][] endpoints, long[][][] pairwise) {
        int n=unary.length;
        if(endpoints.length!=pairwise.length) throw new IllegalArgumentException("pair count mismatch");
        long constant=0;
        long[] oneCost=new long[n];
        for(int i=0;i<n;i++) {
            if(unary[i].length!=2) throw new IllegalArgumentException("two unary costs required");
            constant=Math.addExact(constant,unary[i][0]);
            oneCost[i]=Math.subtractExact(unary[i][1],unary[i][0]);
        }
        long[][] pairCapacity=new long[endpoints.length][3];
        for(int e=0;e<endpoints.length;e++) {
            if(endpoints[e].length!=2 || pairwise[e].length!=2
                    || pairwise[e][0].length!=2 || pairwise[e][1].length!=2)
                throw new IllegalArgumentException("pairwise table must be 2 by 2");
            int u=endpoints[e][0],v=endpoints[e][1];
            if(u<0||u>=n||v<0||v>=n||u==v) throw new IllegalArgumentException("invalid pair");
            long e00=pairwise[e][0][0],e01=pairwise[e][0][1];
            long e10=pairwise[e][1][0],e11=pairwise[e][1][1];
            long capacity=Math.subtractExact(Math.addExact(e01,e10),Math.addExact(e00,e11));
            if(capacity<0) throw new IllegalArgumentException("pairwise energy is not submodular");
            constant=Math.addExact(constant,e00);
            oneCost[u]=Math.addExact(oneCost[u],Math.subtractExact(e10,e00));
            oneCost[v]=Math.addExact(oneCost[v],Math.subtractExact(e11,e10));
            pairCapacity[e][0]=u;pairCapacity[e][1]=v;pairCapacity[e][2]=capacity;
        }
        int source=n,sink=n+1;
        Dinic flow=new Dinic(n+2);
        for(int i=0;i<n;i++) {
            if(oneCost[i]>=0) flow.add(source,i,oneCost[i]);
            else {
                constant=Math.addExact(constant,oneCost[i]);
                flow.add(i,sink,Math.negateExact(oneCost[i]));
            }
        }
        for(long[] edge:pairCapacity) flow.add((int)edge[0],(int)edge[1],edge[2]);
        long cut=flow.maxFlow(source,sink);
        boolean[] reachable=flow.reachable(source),labelOne=new boolean[n];
        for(int i=0;i<n;i++) labelOne[i]=!reachable[i];
        return new Result(Math.addExact(constant,cut),labelOne);
    }

    private static final class Dinic {
        final List<List<Edge>> graph;
        final int[] level,next;
        Dinic(int n) {
            graph=new ArrayList<>();for(int i=0;i<n;i++)graph.add(new ArrayList<>());
            level=new int[n];next=new int[n];
        }
        void add(int from,int to,long capacity) {
            Edge forward=new Edge(to,graph.get(to).size(),capacity);
            Edge reverse=new Edge(from,graph.get(from).size(),0);
            graph.get(from).add(forward);graph.get(to).add(reverse);
```

```
        }
        long maxFlow(int source,int sink) {
            long total=0;
            while(buildLevels(source,sink)) {
                Arrays.fill(next,0);
                long sent;
                while((sent=send(source,sink,Long.MAX_VALUE))>0) total=Math.addExact(total,sent);
            }
            return total;
        }
        boolean buildLevels(int source,int sink) {
            Arrays.fill(level,-1);level[source]=0;
            ArrayDeque<Integer> queue=new ArrayDeque<>();queue.add(source);
            while(!queue.isEmpty()) {
                int vertex=queue.remove();
                for(Edge edge:graph.get(vertex)) if(edge.capacity>0&&level[edge.to]<0) {
                    level[edge.to]=level[vertex]+1;queue.add(edge.to);
                }
            }
            return level[sink]>=0;
        }
        long send(int vertex,int sink,long amount) {
            if(vertex==sink)return amount;
            for(;next[vertex]<graph.get(vertex).size();next[vertex]++) {
                Edge edge=graph.get(vertex).get(next[vertex]);
                if(edge.capacity==0||level[edge.to]!=level[vertex]+1)continue;
                long sent=send(edge.to,sink,Math.min(amount,edge.capacity));
                if(sent==0)continue;
                edge.capacity-=sent;
                Edge reverse=graph.get(edge.to).get(edge.reverse);
                reverse.capacity=Math.addExact(reverse.capacity,sent);
                return sent;
            }
            return 0;
        }
        boolean[] reachable(int source) {
            boolean[] seen=new boolean[graph.size()];
            ArrayDeque<Integer> queue=new ArrayDeque<>();seen[source]=true;queue.add(source);
            while(!queue.isEmpty()) {
                int vertex=queue.remove();
                for(Edge edge:graph.get(vertex))if(edge.capacity>0&&!seen[edge.to]) {
                    seen[edge.to]=true;queue.add(edge.to);
                }
            }
            return seen;
        }
    }
}
```

6 Trees

Euler order, ancestors, and path decomposition exploit the absence of cycles.

6.1 Diameter, Euler tour, heavy-light

Level Advanced
Use when Work with subtree intervals or changing path sums.
Cost O(n) setup; O(log^2 n) path queries.
Watch for Heavy-light setup is recursive and expects a tree.
Dependencies graphs, structures

Core idea. A tree path has unique structure; Euler order turns subtrees into intervals.
Worked example. Heavy-light decomposition breaks a path into a logarithmic number of chains.
Practice. Compare path sums against a simple parent walk on small random trees.

```
package notebook;

import java.util.ArrayDeque;
import java.util.Arrays;
import java.util.List;

/** Algorithms for connected, undirected trees with vertices 0..n-1. */
public final class TreeAlgorithms {
    private TreeAlgorithms() {}

    /** Diameter length in edges. O(n). */
    public static int diameter(List<List<Integer>> tree) {
        if (tree.isEmpty()) return 0;
        int endpoint = farthest(tree, 0)[0];
        return farthest(tree, endpoint)[1];
    }

    private static int[] farthest(List<List<Integer>> tree, int start) {
        int[] dist = Graphs.bfs(tree, start);
        int best = start;
        for (int i = 0; i < dist.length; i++) if (dist[i] > dist[best]) best = i;
        return new int[] {best, dist[best]};
    }

    /** Euler entry/exit times; subtree(v) is [tin[v], tout[v]). O(n). */
    public static final class EulerTour {
        public final int[] tin, tout, order;
        public EulerTour(List<List<Integer>> tree, int root) {
            int n = tree.size();
            tin = new int[n]; tout = new int[n]; order = new int[n];
            Arrays.fill(tin, -1);
            ArrayDeque<int[]> stack = new ArrayDeque<>();
            stack.push(new int[] {root, -1, 0});
            int timer = 0;
            while (!stack.isEmpty()) {
                int[] frame = stack.peek();
                int v = frame[0], parent = frame[1];
                if (frame[2] == 0) { tin[v] = timer; order[timer++] = v; }
                if (frame[2] == tree.get(v).size()) { tout[v] = timer; stack.pop(); continue; }
                int to = tree.get(v).get(frame[2]++);
                if (to != parent && tin[to] == -1) stack.push(new int[] {to, v, 0});
            }
            if (timer != n) throw new IllegalArgumentException("disconnected tree");
        }
    }

    /** Point updates and vertex-sum path queries using heavy-light decomposition. O(log^2 n). */
    public static final class HeavyLight {
        private final List<List<Integer>> tree;
        private final int[] parent, depth, size, heavy, head, position;
        private final DataStructures.SegmentTree sums;
        private int clock;

        public HeavyLight(List<List<Integer>> tree, long[] values, int root) {
            if (tree.size() != values.length) throw new IllegalArgumentException();
            this.tree = tree;
            int n = tree.size();
```

```
            parent = new int[n]; depth = new int[n]; size = new int[n];
            heavy = new int[n]; head = new int[n]; position = new int[n];
            Arrays.fill(heavy, -1);
            Arrays.fill(parent, -1);
            buildSizes(root, -1);
            if (size[root] != n) throw new IllegalArgumentException("disconnected tree");
            decompose(root, root);
            long[] ordered = new long[n];
            for (int v = 0; v < n; v++) ordered[position[v]] = values[v];
            sums = new DataStructures.SegmentTree(ordered);
        }

        private void buildSizes(int v, int p) {
            parent[v] = p; size[v] = 1;
            int largest = 0;
            for (int to : tree.get(v)) if (to != p) {
                depth[to] = depth[v] + 1;
                buildSizes(to, v);
                size[v] += size[to];
                if (size[to] > largest) { largest = size[to]; heavy[v] = to; }
            }
        }

        private void decompose(int v, int chainHead) {
            head[v] = chainHead; position[v] = clock++;
            if (heavy[v] != -1) decompose(heavy[v], chainHead);
            for (int to : tree.get(v)) if (to != parent[v] && to != heavy[v]) decompose(to, to);
        }

        public void set(int vertex, long value) { sums.set(position[vertex], value); }

        public long pathSum(int a, int b) {
            long result = 0;
            while (head[a] != head[b]) {
                if (depth[head[a]] < depth[head[b]]) { int tmp = a; a = b; b = tmp; }
                result += sums.sum(position[head[a]], position[a] + 1);
                a = parent[head[a]];
            }
            if (depth[a] > depth[b]) { int tmp = a; a = b; b = tmp; }
            return result + sums.sum(position[a], position[b] + 1);
        }
    }
}
```

7 Strings

Choose a one-pattern, many-pattern, palindrome, or suffix method.

7.1 Characters, frequencies, and direct matching

Level Beginner
Use when Count letters, test palindromes, or match a short pattern.
Cost O(n) scans; O(nm) direct matching.
Watch for Java char values are UTF-16 code units; the frequency helper accepts a-z only.

Core idea. A string is an indexed sequence. Start with direct comparisons before choosing a faster string algorithm.
Worked example. The pattern aa occurs three times in aaaa because matches can overlap.
Practice. Test an empty text, one character, and overlapping matches.

```
package notebook;

/** Character and substring patterns for introductory contest problems. */
public final class BeginnerStrings {
```

```
    private BeginnerStrings() {}

    /** Compare characters from both ends. O(n) time, O(1) extra space. */
    public static boolean isPalindrome(String text) {
        for (int left = 0, right = text.length() - 1; left < right; left++, right--)
            if (text.charAt(left) != text.charAt(right)) return false;
        return true;
    }

    /** Count lowercase English letters. O(n) time, O(1) space; rejects other characters. */
    public static int[] lowercaseFrequencies(String text) {
        int[] frequency = new int[26];
        for (int i = 0; i < text.length(); i++) {
            char ch = text.charAt(i);
            if (ch < 'a' || ch > 'z') throw new IllegalArgumentException("expected a-z");
            frequency[ch - 'a']++;
        }
        return frequency;
    }

    /** Count exact occurrences, including overlaps. O(n*m) time. */
    public static int countOccurrences(String text, String pattern) {
        if (pattern.isEmpty()) throw new IllegalArgumentException("pattern must not be empty");
        int count = 0;
        for (int start = 0; start + pattern.length() <= text.length(); start++) {
            int matched = 0;
            while (matched < pattern.length() && text.charAt(start + matched) == pattern.charAt(matched))
                matched++;
            if (matched == pattern.length()) count++;
        }
        return count;
    }

    /** Longest shared prefix of two strings. O(min(n,m)) time. */
    public static String commonPrefix(String first, String second) {
        int length = 0;
        while (length < first.length() && length < second.length()
                && first.charAt(length) == second.charAt(length)) length++;
        return first.substring(0, length);
    }
}
```

7.2 KMP, Manacher, Boyer-Moore, Z

Level Intermediate
Use when Search one pattern or find the longest palindrome.
Cost Linear for KMP, Manacher, and Z.
Watch for Uses Java UTF-16 code units.

Core idea. Prefix, Z, and palindrome arrays summarize repeated character comparisons.
Worked example. The prefix function at i gives the longest proper border of the prefix ending at i.
Practice. Compare matches with direct substring search on random short strings.

```
package notebook;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** String algorithms operate on Java UTF-16 code units. */
public final class StringsAlgorithms {
    private StringsAlgorithms() {}

    /** KMP match starts, including all n+1 positions for an empty pattern. O(n+m). */
    public static List<Integer> kmp(String text, String pattern) {
        List<Integer> matches = new ArrayList<>();
        if (pattern.isEmpty()) {
            for (int i = 0; i <= text.length(); i++) matches.add(i);
```

```
            return matches;
        }
        int[] pi = prefixFunction(pattern);
        for (int i = 0, j = 0; i < text.length(); i++) {
            while (j > 0 && text.charAt(i) != pattern.charAt(j)) j = pi[j - 1];
            if (text.charAt(i) == pattern.charAt(j)) j++;
            if (j == pattern.length()) { matches.add(i - j + 1); j = pi[j - 1]; }
        }
        return matches;
    }

    /** Prefix function. O(n). */
    public static int[] prefixFunction(String s) {
        int[] pi = new int[s.length()];
        for (int i = 1; i < s.length(); i++) {
            int j = pi[i - 1];
            while (j > 0 && s.charAt(i) != s.charAt(j)) j = pi[j - 1];
            if (s.charAt(i) == s.charAt(j)) j++;
            pi[i] = j;
        }
        return pi;
    }

    /** Longest palindromic substring by Manacher, O(n). */
    public static String manacher(String s) {
        if (s.isEmpty()) return "";
        int n = s.length(), bestStart = 0, bestLen = 1;
        int[] odd = new int[n], even = new int[n];
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = i > r ? 1 : Math.min(odd[l + r - i], r - i + 1);
            while (i - k >= 0 && i + k < n && s.charAt(i - k) == s.charAt(i + k)) k++;
            odd[i] = k--;
            int len = 2 * odd[i] - 1;
            if (len > bestLen) { bestLen = len; bestStart = i - odd[i] + 1; }
            if (i + k > r) { l = i - k; r = i + k; }
        }
        for (int i = 0, l = 0, r = -1; i < n; i++) {
            int k = i > r ? 0 : Math.min(even[l + r - i + 1], r - i + 1);
            while (i - k - 1 >= 0 && i + k < n && s.charAt(i - k - 1) == s.charAt(i + k)) k++;
            even[i] = k--;
            int len = 2 * even[i];
            if (len > bestLen) { bestLen = len; bestStart = i - even[i]; }
            if (i + k > r) { l = i - k - 1; r = i + k; }
        }
        return s.substring(bestStart, bestStart + bestLen);
    }

    /** Boyer-Moore bad-character search. O(nm) worst case; UTF-16 aware. */
    public static List<Integer> boyerMoore(String text, String pattern) {
        List<Integer> matches = new ArrayList<>();
        int n = text.length(), m = pattern.length();
        if (m == 0) {
            for (int i = 0; i <= n; i++) matches.add(i);
            return matches;
        }
        int[] last = new int[Character.MAX_VALUE + 1];
        Arrays.fill(last, -1);
        for (int i = 0; i < m; i++) last[pattern.charAt(i)] = i;
        for (int shift = 0; shift <= n - m;) {
            int j = m - 1;
            while (j >= 0 && pattern.charAt(j) == text.charAt(shift + j)) j--;
            if (j < 0) {
                matches.add(shift);
                shift += shift + m < n ? m - last[text.charAt(shift + m)] : 1;
            } else shift += Math.max(1, j - last[text.charAt(shift + j)]);
        }
        return matches;
    }

    /** Z-function: z[i] is longest common prefix of s and s.substring(i). O(n). */
    public static int[] zFunction(String s) {
        int[] z = new int[s.length()];
```

```
        for (int i = 1, l = 0, r = 0; i < s.length(); i++) {
            if (i <= r) z[i] = Math.min(r - i + 1, z[i - l]);
            while (i + z[i] < s.length() && s.charAt(z[i]) == s.charAt(i + z[i])) z[i]++;
            if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
        }
        return z;
    }
}
```

7.3 Trie and rolling hash

Level Intermediate
Use when Store prefixes or compare substrings quickly.
Cost O(total characters) build; O(1) hash query.
Watch for Hash equality is probabilistic.

Core idea. A trie shares common prefixes; rolling hash compresses a substring into modular values.
Worked example. Words car and cat share the c-a prefix nodes in a trie.
Practice. Use direct string comparison when a hash collision would be unacceptable.

```
package notebook;

import java.util.HashMap;
import java.util.Map;

/** Trie and double rolling hash over Java UTF-16 code units. */
public final class AdvancedStrings {
    private AdvancedStrings() {}

    public static final class Trie {
        private static final class Node {
            Map<Character, Node> children = new HashMap<>();
            int terminalCount;
        }
        private final Node root = new Node();

        /** Insert a word, including the empty word. O(length). */
        public void add(String word) {
            Node node = root;
            for (int i = 0; i < word.length(); i++)
                node = node.children.computeIfAbsent(word.charAt(i), key -> new Node());
            node.terminalCount++;
        }

        public int count(String word) {
            Node node = root;
            for (int i = 0; i < word.length(); i++) {
                node = node.children.get(word.charAt(i));
                if (node == null) return 0;
            }
            return node.terminalCount;
        }

        public boolean hasPrefix(String prefix) {
            Node node = root;
            for (int i = 0; i < prefix.length(); i++) {
                node = node.children.get(prefix.charAt(i));
                if (node == null) return false;
            }
            return true;
        }
    }

    /** O(n) build, O(1) half-open substring hash. Hash equality is probabilistic. */
    public static final class RollingHash {
        private static final long MOD1 = 1_000_000_007L, MOD2 = 1_000_000_009L;
        private static final long BASE = 911_382_323L;
        private final long[] h1, h2, p1, p2;
```

```
        public RollingHash(String s) {
            int n = s.length();
            h1 = new long[n + 1]; h2 = new long[n + 1];
            p1 = new long[n + 1]; p2 = new long[n + 1];
            p1[0] = p2[0] = 1;
            for (int i = 0; i < n; i++) {
                long c = s.charAt(i) + 1L;
                h1[i + 1] = (h1[i] * BASE + c) % MOD1;
                h2[i + 1] = (h2[i] * BASE + c) % MOD2;
                p1[i + 1] = p1[i] * BASE % MOD1;
                p2[i + 1] = p2[i] * BASE % MOD2;
            }
        }
        public long hash(int left, int rightExclusive) {
            if (left < 0 || left > rightExclusive || rightExclusive >= h1.length)
                throw new IndexOutOfBoundsException();
            int len = rightExclusive - left;
            long a = (h1[rightExclusive] - h1[left] * p1[len] % MOD1 + MOD1) % MOD1;
            long b = (h2[rightExclusive] - h2[left] * p2[len] % MOD2 + MOD2) % MOD2;
            return (a << 32) | b;
        }
    }
}
```

7.4 Suffix array and LCP

Level Advanced
Use when Order suffixes or count distinct substrings.
Cost O(n log^2 n) array; O(n) LCP.
Watch for Comparison sorting is simple but slower.

Core idea. Sorting suffixes turns substring questions into adjacent comparisons.
Worked example. Every repeated substring appears as a prefix of at least two neighboring suffixes.
Practice. Compare distinct-substring counts against a set of all substrings for short text.

```
package notebook;

import java.util.Arrays;

/** Suffix array and LCP over UTF-16 code units. */
public final class SuffixAlgorithms {
    private SuffixAlgorithms() {}

    /** Suffix starts in lexicographic order. O(n log^2 n) with comparison sorting. */
    public static int[] suffixArray(String s) {
        int n = s.length();
        Integer[] order = new Integer[n];
        int[] rank = new int[n], nextRank = new int[n];
        for (int i = 0; i < n; i++) { order[i] = i; rank[i] = s.charAt(i); }
        for (int width = 1; width < n; width <<= 1) {
            final int[] current = rank;
            final int step = width;
            Arrays.sort(order, (a, b) -> {
                if (current[a] != current[b]) return Integer.compare(current[a], current[b]);
                return Integer.compare(a + step < n ? current[a + step] : -1,
                                       b + step < n ? current[b + step] : -1);
            });
            nextRank[order[0]] = 0;
            for (int i = 1; i < n; i++) {
                int a = order[i - 1], b = order[i];
                boolean different = current[a] != current[b]
                    || (a + step < n ? current[a + step] : -1)
                    != (b + step < n ? current[b + step] : -1);
                nextRank[b] = nextRank[a] + (different ? 1 : 0);
            }
            int[] swap = rank; rank = nextRank; nextRank = swap;
            if (rank[order[n - 1]] == n - 1) break;
            if (width > n / 2) break;
```

```
        }
        int[] result = new int[n];
        for (int i = 0; i < n; i++) result[i] = order[i];
        return result;
    }

    /** lcp[i] = common prefix length of suffixes sa[i] and sa[i-1]; lcp[0]=0. O(n). */
    public static int[] lcp(String s, int[] sa) {
        int n = s.length();
        if (sa.length != n) throw new IllegalArgumentException();
        int[] rank = new int[n], lcp = new int[n];
        for (int i = 0; i < n; i++) rank[sa[i]] = i;
        for (int i = 0, common = 0; i < n; i++) {
            if (rank[i] == 0) { common = 0; continue; }
            int j = sa[rank[i] - 1];
            while (i + common < n && j + common < n
                    && s.charAt(i + common) == s.charAt(j + common)) common++;
            lcp[rank[i]] = common;
            if (common > 0) common--;
        }
        return lcp;
    }

    /** Number of distinct nonempty substrings. O(n log^2 n). */
    public static long distinctSubstrings(String s) {
        int[] sa = suffixArray(s), lcp = lcp(s, sa);
        long count = 0;
        for (int i = 0; i < sa.length; i++) count += s.length() - sa[i] - lcp[i];
        return count;
    }
}
```

7.5 Aho-Corasick

Level Advanced
Use when Match many patterns against text.
Cost Linear in text and automaton size.
Watch for Lowercase a-z only.

Core idea. Failure links let multiple pattern searches share work while scanning text once.
Worked example. Patterns he and she can both finish during a scan of she.
Practice. Test duplicate patterns and patterns that are suffixes of other patterns.

```
package notebook;

import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/** Multi-pattern search for lowercase ASCII strings. O(total pattern length + text length * alphabet). */
public final class AhoCorasick {
    private static final class Node {
        int[] next = new int[26];
        int fail, outputs;
        Node() { Arrays.fill(next, -1); }
    }
    private final List<Node> nodes = new ArrayList<>();
    private boolean built;

    public AhoCorasick() { nodes.add(new Node()); }

    /** Add a nonempty lowercase pattern before build(). */
    public void add(String pattern) {
        if (built || pattern.isEmpty()) throw new IllegalArgumentException();
        int state = 0;
        for (int i = 0; i < pattern.length(); i++) {
            int c = index(pattern.charAt(i));
```

```
            if (nodes.get(state).next[c] == -1) {
                nodes.get(state).next[c] = nodes.size();
                nodes.add(new Node());
            }
            state = nodes.get(state).next[c];
        }
        nodes.get(state).outputs++;
    }

    public void build() {
        if (built) return;
        ArrayDeque<Integer> queue = new ArrayDeque<>();
        for (int c = 0; c < 26; c++) {
            int to = nodes.get(0).next[c];
            if (to == -1) nodes.get(0).next[c] = 0;
            else queue.add(to);
        }
        while (!queue.isEmpty()) {
            int state = queue.remove();
            Node node = nodes.get(state);
            node.outputs += nodes.get(node.fail).outputs;
            for (int c = 0; c < 26; c++) {
                int to = node.next[c];
                if (to == -1) node.next[c] = nodes.get(node.fail).next[c];
                else { nodes.get(to).fail = nodes.get(node.fail).next[c]; queue.add(to); }
            }
        }
        built = true;
    }

    /** Count all occurrences, including overlaps and duplicate patterns. O(text length). */
    public long countMatches(String text) {
        if (!built) build();
        int state = 0;
        long count = 0;
        for (int i = 0; i < text.length(); i++) {
            state = nodes.get(state).next[index(text.charAt(i))];
            count += nodes.get(state).outputs;
        }
        return count;
    }

    private static int index(char c) {
        if (c < 'a' || c > 'z') throw new IllegalArgumentException("lowercase a-z only");
        return c - 'a';
    }
}
```

7.6 Minimum cyclic rotation

Level Intermediate
Use when Find a lexicographically smallest rotation.
Cost O(n) time.
Watch for Uses Java UTF-16 code units.

Core idea. Booth discards candidate starts after each mismatch.
Worked example. The minimum rotation of bba is abb.
Practice. Compare all rotations on short strings.

```
package notebook;

/** Lexicographically minimum cyclic rotation by Booth's linear-time algorithm. */
public final class StringExtensions {
    private StringExtensions() {}

    /** Index of the smallest rotation; zero for empty text. O(n) time, O(1) extra space. */
    public static int minimumRotationIndex(String text) {
        int n = text.length();
```

```
        if (n == 0) return 0;
        int first = 0, second = 1, offset = 0;
        while (first < n && second < n && offset < n) {
            char a = text.charAt((first + offset) % n);
            char b = text.charAt((second + offset) % n);
            if (a == b) { offset++; continue; }
            if (a > b) first += offset + 1;
            else second += offset + 1;
            if (first == second) second++;
            offset = 0;
        }
        return Math.min(first, second);
    }

    /** Lexicographically smallest cyclic rotation. O(n). */
    public static String minimumRotation(String text) {
        int index = minimumRotationIndex(text);
        return text.substring(index) + text.substring(0, index);
    }
}
```

7.7 Suffix automaton

Level Advanced
Use when Count distinct substrings or find a longest common substring.
Cost Expected O(n) build.
Watch for HashMap transitions use UTF-16 code units.

Core idea. Clone states preserve suffix links when an equivalence class splits.
Worked example. The string aba has five distinct nonempty substrings.
Practice. Compare against a set of all short substrings.

```
package notebook;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/** Suffix automaton over Java UTF-16 code units. Expected O(n) build time. */
public final class SuffixAutomaton {
    private static final class State {
        int length, link = -1;
        Map<Character, Integer> next = new HashMap<>();
    }
    private final List<State> states = new ArrayList<>();
    private int last;

    public SuffixAutomaton(String text) {
        states.add(new State());
        for (int i = 0; i < text.length(); i++) extend(text.charAt(i));
    }

    private void extend(char symbol) {
        int current = states.size();
        State state = new State(); state.length = states.get(last).length + 1;
        states.add(state);
        int previous = last;
        while (previous != -1 && !states.get(previous).next.containsKey(symbol)) {
            states.get(previous).next.put(symbol, current);
            previous = states.get(previous).link;
        }
        if (previous == -1) state.link = 0;
        else {
            int target = states.get(previous).next.get(symbol);
            if (states.get(previous).length + 1 == states.get(target).length) state.link = target;
            else {
                int cloneIndex = states.size();
```

```
                State clone = new State();
                clone.length = states.get(previous).length + 1;
                clone.link = states.get(target).link;
                clone.next.putAll(states.get(target).next);
                states.add(clone);
                while (previous != -1 && Integer.valueOf(target).equals(states.get(previous).next.get(symbol))) {
                    states.get(previous).next.put(symbol, cloneIndex);
                    previous = states.get(previous).link;
                }
                states.get(target).link = cloneIndex;
                state.link = cloneIndex;
            }
        }
        last = current;
    }

    /** Whether pattern occurs as a substring. Expected O(pattern length). */
    public boolean contains(String pattern) {
        int state = 0;
        for (int i = 0; i < pattern.length(); i++) {
            Integer next = states.get(state).next.get(pattern.charAt(i));
            if (next == null) return false;
            state = next;
        }
        return true;
    }

    /** Number of distinct nonempty substrings. O(number of states). */
    public long distinctSubstringCount() {
        long count = 0;
        for (int i = 1; i < states.size(); i++)
            count += states.get(i).length - states.get(states.get(i).link).length;
        return count;
    }

    /** Length of the longest substring shared with other. Expected O(other length). */
    public int longestCommonSubstringLength(String other) {
        int state = 0, length = 0, best = 0;
        for (int i = 0; i < other.length(); i++) {
            char symbol = other.charAt(i);
            while (state != 0 && !states.get(state).next.containsKey(symbol)) {
                state = states.get(state).link;
                length = states.get(state).length;
            }
            Integer next = states.get(state).next.get(symbol);
            if (next != null) { state = next; length++; }
            else { state = 0; length = 0; }
            best = Math.max(best, length);
        }
        return best;
    }
}
```

8 Number theory and combinatorics

Use exact or modular arithmetic according to the requested result.

8.1 GCD, trial division, divisors, and powers

Level Beginner
Use when Solve small divisibility and integer arithmetic tasks.
Cost GCD O(log n); trial division O(sqrt n); power O(log exponent).
Watch for Exact power reports long overflow; inputs to gcd must be nonnegative.

Core idea. A divisor larger than the square root has a matching smaller divisor. Euclid repeatedly replaces (a,b) by (b,a mod b).
Worked example. The divisors of 36 come in pairs around 6, giving nine positive divisors.
Practice. Compare trial-division results with direct enumeration for small values.

```
package notebook;

/** Elementary integer math before modular arithmetic and number theory. */
public final class BeginnerMath {
    private BeginnerMath() {}

    /** Euclid's algorithm for nonnegative inputs. O(log min(a,b)) time. */
    public static long gcd(long a, long b) {
        if (a < 0 || b < 0) throw new IllegalArgumentException("expected nonnegative values");
        while (b != 0) { long next = a % b; a = b; b = next; }
        return a;
    }

    /** Test primality by trial division. O(sqrt(n)) time, O(1) space. */
    public static boolean isPrime(long value) {
        if (value < 2) return false;
        for (long divisor = 2; divisor <= value / divisor; divisor++)
            if (value % divisor == 0) return false;
        return true;
    }

    /** Count positive divisors of a positive integer. O(sqrt(n)) time. */
    public static long divisorCount(long value) {
        if (value <= 0) throw new IllegalArgumentException("expected a positive value");
        long count = 0;
        for (long divisor = 1; divisor <= value / divisor; divisor++)
            if (value % divisor == 0) count += divisor == value / divisor ? 1 : 2;
        return count;
    }

    /** Compute base^exponent for a nonnegative exponent. O(log exponent) time. */
    public static long power(long base, long exponent) {
        if (exponent < 0) throw new IllegalArgumentException("negative exponent");
        long result = 1;
        while (exponent != 0) {
            if ((exponent & 1) != 0) result = Math.multiplyExact(result, base);
            exponent >>= 1;
            if (exponent != 0) base = Math.multiplyExact(base, base);
        }
        return result;
    }
}
```

8.2 GCD, modular arithmetic, primality, CRT

Level Intermediate
Use when Work with divisibility, primes, congruences, or roots.
Cost Euclid O(log n); sieve O(n log log n).
Watch for Overflow-safe modular multiplication trades speed for range.

Core idea. Choose the correct number representation and modulus before applying arithmetic routines.
Worked example. Euclid finds gcd; the extended form also gives coefficients for modular inverses.
Practice. Compare modular operations against BigInteger on boundary inputs.

```
package notebook;

import java.math.BigInteger;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.function.DoubleUnaryOperator;

/** Integer number theory. Modular operations require mod > 0. */
public final class NumberTheory {
    private NumberTheory() {}

    public static long gcd(long a, long b) {
        if (a == Long.MIN_VALUE || b == Long.MIN_VALUE)
            return BigInteger.valueOf(a).gcd(BigInteger.valueOf(b)).longValueExact();
        a = Math.abs(a); b = Math.abs(b);
        while (b != 0) { long t = a % b; a = b; b = t; }
        return a;
    }

    public static long lcm(long a, long b) {
        if (a == 0 || b == 0) return 0;
        long product = Math.multiplyExact(a / gcd(a, b), b);
        if (product == Long.MIN_VALUE) throw new ArithmeticException("lcm does not fit in long");
        return Math.abs(product);
    }

    /** Returns [gcd(a,b), x, y] with ax+by=gcd; coefficients must fit in long. */
    public static long[] extendedGcd(long a, long b) {
        BigInteger aa = BigInteger.valueOf(a), bb = BigInteger.valueOf(b);
        BigInteger oldR = aa, r = bb, oldS = BigInteger.ONE, s = BigInteger.ZERO;
        BigInteger oldT = BigInteger.ZERO, t = BigInteger.ONE;
        while (r.signum() != 0) {
            BigInteger q = oldR.divide(r);
            BigInteger nr = oldR.subtract(q.multiply(r)); oldR = r; r = nr;
            BigInteger ns = oldS.subtract(q.multiply(s)); oldS = s; s = ns;
            BigInteger nt = oldT.subtract(q.multiply(t)); oldT = t; t = nt;
        }
        if (oldR.signum() < 0) { oldR = oldR.negate(); oldS = oldS.negate(); oldT = oldT.negate(); }
        return new long[] { oldR.longValueExact(), oldS.longValueExact(), oldT.longValueExact() };
    }

    /** Exact multiplication modulo mod without signed-long overflow. O(log b). */
    public static long modMul(long a, long b, long mod) {
        if (mod <= 0) throw new IllegalArgumentException("mod must be positive");
        long x = Math.floorMod(a, mod), y = Math.floorMod(b, mod), result = 0;
        while (y > 0) {
            if ((y & 1) != 0) result = addMod(result, x, mod);
            y >>>= 1;
            if (y > 0) x = addMod(x, x, mod);
        }
        return result;
    }

    private static long addMod(long a, long b, long mod) {
        return a >= mod - b ? a - (mod - b) : a + b;
    }

    /** a^exponent modulo mod. O(log exponent * log mod). */
```

```
    public static long modPow(long a, long exponent, long mod) {
        if (exponent < 0 || mod <= 0) throw new IllegalArgumentException();
        long result = 1 % mod, base = Math.floorMod(a, mod);
        while (exponent > 0) {
            if ((exponent & 1) != 0) result = modMul(result, base, mod);
            exponent >>>= 1;
            if (exponent > 0) base = modMul(base, base, mod);
        }
        return result;
    }

    /** Deterministic Miller-Rabin for signed 64-bit positive integers. */
    public static boolean isPrime(long n) {
        if (n < 2) return false;
        for (long p : new long[] {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}) {
            if (n % p == 0) return n == p;
        }
        long d = n - 1;
        int s = Long.numberOfTrailingZeros(d);
        d >>= s;
        for (long witness : new long[] {2, 325, 9375, 28178, 450775, 9780504, 1795265022}) {
            long a = witness % n;
            if (a == 0) continue;
            long x = modPow(a, d, n);
            if (x == 1 || x == n - 1) continue;
            boolean passed = false;
            for (int r = 1; r < s; r++) {
                x = modMul(x, x, n);
                if (x == n - 1) { passed = true; break; }
            }
            if (!passed) return false;
        }
        return true;
    }

    /** Primes <= n; O(n log log n) time, O(n) space. */
    public static List<Integer> sieve(int n) {
        if (n < 2) return new ArrayList<>();
        boolean[] composite = new boolean[n + 1];
        for (int p = 2; (long) p * p <= n; p++)
            if (!composite[p]) for (int x = p * p; x <= n; x += p) composite[x] = true;
        List<Integer> primes = new ArrayList<>();
        for (int i = 2; i <= n; i++) if (!composite[i]) primes.add(i);
        return primes;
    }

    /** Sorted positive divisors of n; n > 0. O(sqrt n). */
    public static List<Long> divisors(long n) {
        if (n <= 0) throw new IllegalArgumentException();
        List<Long> result = new ArrayList<>();
        for (long d = 1; d <= n / d; d++) {
            if (n % d == 0) {
                result.add(d);
                if (d != n / d) result.add(n / d);
            }
        }
        Collections.sort(result);
        return result;
    }

    /** Exact factorial; n >= 0. */
    public static BigInteger factorial(int n) {
        if (n < 0) throw new IllegalArgumentException();
        BigInteger result = BigInteger.ONE;
        for (int i = 2; i <= n; i++) result = result.multiply(BigInteger.valueOf(i));
        return result;
    }

    /** Generalized CRT. Returns [least nonnegative solution, lcm], or null if inconsistent. */
    public static BigInteger[] chineseRemainder(long[] moduli, long[] residues) {
        if (moduli.length != residues.length) throw new IllegalArgumentException();
        BigInteger x = BigInteger.ZERO, m = BigInteger.ONE;
```

```
        for (int i = 0; i < moduli.length; i++) {
            if (moduli[i] <= 0) throw new IllegalArgumentException("moduli must be positive");
            BigInteger next = BigInteger.valueOf(moduli[i]);
            BigInteger target = BigInteger.valueOf(residues[i]).mod(next);
            BigInteger g = m.gcd(next), delta = target.subtract(x);
            if (!delta.mod(g).equals(BigInteger.ZERO)) return null;
            BigInteger reduced = next.divide(g);
            BigInteger k = reduced.equals(BigInteger.ONE) ? BigInteger.ZERO
                : delta.divide(g).multiply(m.divide(g).modInverse(reduced)).mod(reduced);
            x = x.add(m.multiply(k));
            m = m.multiply(reduced);
            x = x.mod(m);
        }
        return new BigInteger[] { x, m };
    }

    /** Newton iteration; derivative must remain nonzero near the root. */
    public static double newtonRaphson(DoubleUnaryOperator f, DoubleUnaryOperator derivative,
                                       double start, double tolerance, int maxIterations) {
        if (!(tolerance > 0) || maxIterations <= 0) throw new IllegalArgumentException();
        double x = start;
        for (int i = 0; i < maxIterations; i++) {
            double fx = f.applyAsDouble(x), dx = derivative.applyAsDouble(x);
            if (Math.abs(fx) <= tolerance) return x;
            if (!Double.isFinite(dx) || dx == 0) throw new ArithmeticException("zero derivative");
            x -= fx / dx;
            if (!Double.isFinite(x)) throw new ArithmeticException("non-finite iterate");
        }
        throw new ArithmeticException("did not converge");
    }
}
```

8.3 Factors, phi, inverse, modular nCr, matrices

Level Advanced
Use when Factor integers or exponentiate recurrences.
Cost O(sqrt n) factorization; O(k^3 log exponent) matrices.
Watch for PrimeCombinations needs a prime modulus greater than its limit.
Dependencies math

Core idea. Precomputation pays off when many factor or binomial queries share the same limits.
Worked example. A smallest-prime-factor table decomposes 84 as 2 squared times 3 times 7.
Practice. Compare factorization and totient against direct small-number implementations.

```
package notebook;

import java.math.BigInteger;
import java.util.ArrayList;
import java.util.List;

/** Additional arithmetic routines. Integer overflow is rejected where practical. */
public final class NumberTheoryAdvanced {
    private NumberTheoryAdvanced() {}

    /** Smallest prime factor for every value in [2,n]. O(n log log n). */
    public static int[] smallestPrimeFactors(int n) {
        if (n < 0) throw new IllegalArgumentException();
        int[] spf = new int[n + 1];
        for (int i = 2; i <= n; i++) if (spf[i] == 0) {
            spf[i] = i;
            if ((long) i * i <= n)
                for (int j = i * i; j <= n; j += i) if (spf[j] == 0) spf[j] = i;
        }
        return spf;
    }

    /** Prime factors with multiplicity, ascending. Trial division; O(sqrt n). */
```

```
    public static List<Long> primeFactors(long n) {
        if (n <= 0) throw new IllegalArgumentException();
        List<Long> factors = new ArrayList<>();
        for (long p = 2; p <= n / p; p += p == 2 ? 1 : 2)
            while (n % p == 0) { factors.add(p); n /= p; }
        if (n > 1) factors.add(n);
        return factors;
    }

    /** Euler phi(n), n >= 1. O(sqrt n). */
    public static long phi(long n) {
        if (n <= 0) throw new IllegalArgumentException();
        long result = n, remaining = n;
        for (long p = 2; p <= remaining / p; p++) if (remaining % p == 0) {
            while (remaining % p == 0) remaining /= p;
            result -= result / p;
        }
        if (remaining > 1) result -= result / remaining;
        return result;
    }

    /** Modular inverse for coprime a and mod; throws otherwise. */
    public static long modInverse(long a, long mod) {
        if (mod <= 1) throw new IllegalArgumentException();
        return BigInteger.valueOf(a).mod(BigInteger.valueOf(mod))
            .modInverse(BigInteger.valueOf(mod)).longValueExact();
    }

    /** O(limit) setup, O(1) binomial queries; requires prime mod and limit < mod. */
    public static final class PrimeCombinations {
        private final long mod;
        private final long[] factorial, inverseFactorial;
        public PrimeCombinations(int limit, long primeMod) {
            if (limit < 0 || primeMod <= limit || !NumberTheory.isPrime(primeMod))
                throw new IllegalArgumentException("mod must be prime and greater than limit");
            mod = primeMod;
            factorial = new long[limit + 1];
            inverseFactorial = new long[limit + 1];
            factorial[0] = 1;
            for (int i = 1; i <= limit; i++) factorial[i] = NumberTheory.modMul(factorial[i - 1], i, mod);
            inverseFactorial[limit] = NumberTheory.modPow(factorial[limit], mod - 2, mod);
            for (int i = limit; i > 0; i--) inverseFactorial[i - 1] = NumberTheory.modMul(inverseFactorial[i], i,
     mod);
        }
        public long choose(int n, int r) {
            if (n < 0 || n >= factorial.length) throw new IndexOutOfBoundsException();
            if (r < 0 || r > n) return 0;
            return NumberTheory.modMul(factorial[n],
                NumberTheory.modMul(inverseFactorial[r], inverseFactorial[n - r], mod), mod);
        }
    }

    /** Square matrix exponentiation modulo mod. O(size^3 log exponent). */
    public static long[][] matrixPower(long[][] matrix, long exponent, long mod) {
        if (exponent < 0 || mod <= 0) throw new IllegalArgumentException();
        int n = matrix.length;
        long[][] result = new long[n][n], base = new long[n][n];
        for (int i = 0; i < n; i++) {
            if (matrix[i].length != n) throw new IllegalArgumentException("square matrix required");
            result[i][i] = 1 % mod;
            for (int j = 0; j < n; j++) base[i][j] = Math.floorMod(matrix[i][j], mod);
        }
        while (exponent > 0) {
            if ((exponent & 1) != 0) result = multiply(result, base, mod);
            exponent >>>= 1;
            if (exponent > 0) base = multiply(base, base, mod);
        }
        return result;
    }

    private static long[][] multiply(long[][] a, long[][] b, long mod) {
        int n = a.length;
```

```
        long[][] product = new long[n][n];
        for (int i = 0; i < n; i++) for (int k = 0; k < n; k++)
            for (int j = 0; j < n; j++) {
                long term = NumberTheory.modMul(a[i][k], b[k][j], mod);
                product[i][j] = product[i][j] >= mod - term
                    ? product[i][j] - (mod - term) : product[i][j] + term;
            }
        return product;
    }
}
```

8.4 Exact binomial and Catalan numbers

Level Intermediate
Use when Compute counts beyond long range.
Cost O(n) BigInteger operations.
Watch for BigInteger is exact but allocates memory.

Core idea. BigInteger keeps exact counts when the answer does not fit a primitive type.
Worked example. The number of ways to choose 2 from 5 is 10.
Practice. Check symmetry C(n,k) = C(n,n-k).

```
package notebook;

import java.math.BigInteger;

public final class Combinatorics {
    private Combinatorics() {}

    /** Exact binomial coefficient. O(min(r,n-r)) big-integer operations. */
    public static BigInteger binomial(int n, int r) {
        if (r < 0 || r > n) return BigInteger.ZERO;
        r = Math.min(r, n - r);
        BigInteger ans = BigInteger.ONE;
        for (int i = 1; i <= r; i++)
            ans = ans.multiply(BigInteger.valueOf(n - i + 1)).divide(BigInteger.valueOf(i));
        return ans;
    }

    /** Catalan numbers C[0..n], exact. O(n) big-integer operations. */
    public static BigInteger[] catalan(int n) {
        if (n < 0) throw new IllegalArgumentException();
        BigInteger[] c = new BigInteger[n + 1];
        c[0] = BigInteger.ONE;
        for (int i = 1; i <= n; i++)
            c[i] = c[i - 1].multiply(BigInteger.valueOf(4L * i - 2)).divide(BigInteger.valueOf(i + 1));
        return c;
    }
}
```

8.5 Discrete log, modular root, Mobius

Level Advanced
Use when Solve multiplicative congruences and inversion problems.
Cost O(sqrt modulus) discrete log; O(n) Mobius sieve.
Watch for Discrete log needs a coprime base; roots need a prime modulus.

Core idea. Baby-step giant-step trades memory for speed; Tonelli-Shanks extracts roots.
Worked example. Modulo 7, 3 squared equals 2.
Practice. Compare against exhaustive small powers and squares.

```
package notebook;
```

```java
import java.math.BigInteger;
import java.util.HashMap;
import java.util.Map;

/** Discrete logarithm, modular square root, and the Mobius function. */
public final class ModularExtras {
    private ModularExtras() {}

    /** Smallest x >= 0 with base^x = target (mod modulus), or -1. Requires coprime base and modulus. */
    public static long discreteLog(long base, long target, long modulus) {
        if (modulus <= 1 || modulus > 2_000_000_000L) throw new IllegalArgumentException("modulus out of range");
        base = Math.floorMod(base, modulus); target = Math.floorMod(target, modulus);
        if (BigInteger.valueOf(base).gcd(BigInteger.valueOf(modulus)).longValue() != 1)
            throw new IllegalArgumentException("base and modulus must be coprime");
        long step = (long) Math.ceil(Math.sqrt(modulus));
        Map<Long, Long> baby = new HashMap<>();
        long value = 1;
        for (long j = 0; j < step; j++) {
            baby.putIfAbsent(value, j);
            value = value * base % modulus;
        }
        long factor = BigInteger.valueOf(value).modInverse(BigInteger.valueOf(modulus)).longValue();
        long giant = target;
        for (long i = 0; i <= step; i++) {
            Long j = baby.get(giant);
            if (j != null) return i * step + j;
            giant = giant * factor % modulus;
        }
        return -1;
    }

    /** One square root of value modulo an odd prime, or -1 if none exists. Tonelli-Shanks. */
    public static long modularSquareRoot(long value, long prime) {
        if (prime == 2) return Math.floorMod(value, 2);
        if (prime <= 2 || (prime & 1) == 0 || prime > 2_000_000_000L)
            throw new IllegalArgumentException("expected an odd prime at most 2e9");
        value = Math.floorMod(value, prime);
        if (value == 0) return 0;
        if (power(value, (prime - 1) / 2, prime) != 1) return -1;
        if (prime % 4 == 3) return power(value, (prime + 1) / 4, prime);
        long odd = prime - 1;
        int twos = 0;
        while ((odd & 1) == 0) { odd >>= 1; twos++; }
        long nonResidue = 2;
        while (power(nonResidue, (prime - 1) / 2, prime) != prime - 1) nonResidue++;
        long c = power(nonResidue, odd, prime), x = power(value, (odd + 1) / 2, prime);
        long t = power(value, odd, prime);
        int m = twos;
        while (t != 1) {
            int i = 1;
            long squared = t * t % prime;
            while (i < m && squared != 1) { squared = squared * squared % prime; i++; }
            if (i == m) throw new IllegalStateException("modulus may not be prime");
            long b = power(c, 1L << (m - i - 1), prime);
            x = x * b % prime;
            t = t * b % prime * b % prime;
            c = b * b % prime;
            m = i;
        }
        return x;
    }

    private static long power(long base, long exponent, long modulus) {
        long result = 1;
        while (exponent != 0) {
            if ((exponent & 1) != 0) result = result * base % modulus;
            base = base * base % modulus;
            exponent >>= 1;
        }
        return result;
    }
```

```
    /** Mobius values mu[0..limit] via a linear sieve. O(limit) time and space. */
    public static int[] mobius(int limit) {
        if (limit < 0) throw new IllegalArgumentException();
        int[] mu = new int[limit + 1];
        if (limit == 0) return mu;
        mu[1] = 1;
        int[] primes = new int[limit + 1];
        boolean[] composite = new boolean[limit + 1];
        int count = 0;
        for (int value = 2; value <= limit; value++) {
            if (!composite[value]) { primes[count++] = value; mu[value] = -1; }
            for (int i = 0; i < count && (long) value * primes[i] <= limit; i++) {
                int next = value * primes[i]; composite[next] = true;
                if (value % primes[i] == 0) { mu[next] = 0; break; }
                mu[next] = -mu[value];
            }
        }
        return mu;
    }
}
```

8.6 Derangements, Stirling, partitions, Burnside

Level Intermediate
Use when Count permutations, set partitions, and necklaces.
Cost O(n) to O(n sqrt n) BigInteger work.
Watch for Exact answers can become very large.

Core idea. Recurrences and Burnside count objects without enumerating them.
Worked example. There are 44 derangements of five items.
Practice. Enumerate tiny objects and compare counts.

```
package notebook;

import java.math.BigInteger;

/** Exact counting formulas for permutations, partitions, and rotational symmetry. */
public final class CombinatoricsExtras {
    private CombinatoricsExtras() {}

    /** Derangements of n labeled items. O(n) BigInteger operations. */
    public static BigInteger derangements(int n) {
        if (n < 0) throw new IllegalArgumentException();
        if (n == 0) return BigInteger.ONE;
        BigInteger previous = BigInteger.ONE, current = BigInteger.ZERO;
        for (int i = 2; i <= n; i++) {
            BigInteger next = previous.add(current).multiply(BigInteger.valueOf(i - 1));
            previous = current; current = next;
        }
        return current;
    }

    /** Stirling number S(n,k): partitions into exactly k nonempty unlabeled groups. O(n*k). */
    public static BigInteger stirlingSecond(int n, int k) {
        if (n < 0 || k < 0) throw new IllegalArgumentException();
        if (k > n) return BigInteger.ZERO;
        BigInteger[] dp = new BigInteger[k + 1];
        for (int i = 0; i <= k; i++) dp[i] = BigInteger.ZERO;
        dp[0] = BigInteger.ONE;
        for (int value = 1; value <= n; value++)
            for (int groups = Math.min(value, k); groups >= 0; groups--)
                dp[groups] = groups == 0 ? BigInteger.ZERO
                        : dp[groups - 1].add(dp[groups].multiply(BigInteger.valueOf(groups)));
        return dp[k];
    }

    /** Partition numbers p(0)..p(limit) using generalized pentagonal numbers. O(n*sqrt(n)). */
    public static BigInteger[] partitions(int limit) {
```

```
        if (limit < 0) throw new IllegalArgumentException();
        BigInteger[] result = new BigInteger[limit + 1];
        result[0] = BigInteger.ONE;
        for (int n = 1; n <= limit; n++) {
            BigInteger sum = BigInteger.ZERO;
            for (long k = 1; k * (3 * k - 1) / 2 <= n; k++) {
                int first = (int) (k * (3 * k - 1) / 2);
                int second = (int) (k * (3 * k + 1) / 2);
                if ((k & 1) == 1) {
                    sum = sum.add(result[n - first]);
                    if (second <= n) sum = sum.add(result[n - second]);
                } else {
                    sum = sum.subtract(result[n - first]);
                    if (second <= n) sum = sum.subtract(result[n - second]);
                }
            }
            result[n] = sum;
        }
        return result;
    }

    /** Number of length-n necklaces over colors colors, up to rotation. Burnside in O(n log n). */
    public static BigInteger necklaces(int n, int colors) {
        if (n <= 0 || colors < 0) throw new IllegalArgumentException();
        BigInteger sum = BigInteger.ZERO, base = BigInteger.valueOf(colors);
        for (int shift = 0; shift < n; shift++) sum = sum.add(base.pow(gcd(n, shift)));
        return sum.divide(BigInteger.valueOf(n));
    }

    private static int gcd(int a, int b) {
        while (b != 0) { int next = a % b; a = b; b = next; }
        return a;
    }
}
```

9 Polynomial methods

Convolution accelerates counting and polynomial tasks.

9.1 Number-theoretic transform

Level Advanced
Use when Multiply large polynomials modulo 998244353.
Cost O(n log n).
Watch for Transform length must not exceed 2^23.
Dependencies math

Core idea. A transform changes polynomial multiplication into pointwise multiplication under a suitable prime.
Worked example. Coefficients [1,2] times [3,4] become [3,10,8].
Practice. Compare small random convolutions against the quadratic method.

```
package notebook;

import java.util.Arrays;

/** NTT convolution modulo 998244353 (primitive root 3). O(n log n). */
public final class Convolution {
    public static final int MOD = 998244353;
    private Convolution() {}

    public static int[] multiply(int[] a, int[] b) {
        if (a.length == 0 || b.length == 0) return new int[0];
        int required = a.length + b.length - 1, size = 1;
```

```
        while (size < required) size <<= 1;
        if (size > (1 << 23)) throw new IllegalArgumentException("NTT size exceeds 2^23");
        int[] left = Arrays.copyOf(a, size), right = Arrays.copyOf(b, size);
        for (int i = 0; i < size; i++) {
            left[i] = Math.floorMod(left[i], MOD);
            right[i] = Math.floorMod(right[i], MOD);
        }
        transform(left, false); transform(right, false);
        for (int i = 0; i < size; i++) left[i] = (int) ((long) left[i] * right[i] % MOD);
        transform(left, true);
        return Arrays.copyOf(left, required);
    }

    private static void transform(int[] a, boolean inverse) {
        int n = a.length;
        for (int i = 1, j = 0; i < n; i++) {
            int bit = n >>> 1;
            for (; (j & bit) != 0; bit >>>= 1) j ^= bit;
            j ^= bit;
            if (i < j) { int t = a[i]; a[i] = a[j]; a[j] = t; }
        }
        for (int len = 2; len <= n; len <<= 1) {
            long root = NumberTheory.modPow(3, (MOD - 1) / len, MOD);
            if (inverse) root = NumberTheory.modPow(root, MOD - 2, MOD);
            for (int start = 0; start < n; start += len) {
                long omega = 1;
                for (int j = 0; j < len / 2; j++) {
                    int u = a[start + j];
                    int v = (int) (omega * a[start + j + len / 2] % MOD);
                    a[start + j] = (int) (((long) u + v) % MOD);
                    a[start + j + len / 2] = (u - v + MOD) % MOD;
                    omega = omega * root % MOD;
                }
            }
        }
        if (inverse) {
            long invN = NumberTheory.modPow(n, MOD - 2, MOD);
            for (int i = 0; i < n; i++) a[i] = (int) (a[i] * invN % MOD);
        }
    }
}
```

10 Numerical methods

Solve dense systems, multiply polynomials, and use transforms when their assumptions fit.

10.1 Gaussian elimination and determinant

Level Advanced
Use when Solve dense real systems or compute determinants.
Cost O(n^3) for square matrices.
Watch for Absolute epsilon can misclassify near-singular systems.

Core idea. Pivot on the largest coefficient and eliminate the column.
Worked example. Equations 2x+y=5 and x-y=1 give x=2, y=1.
Practice. Multiply the solution back and check residuals.

```
package notebook;

/** Gaussian elimination for small dense real-valued systems. */
public final class LinearAlgebra {
    private LinearAlgebra() {}
    private static final double EPS = 1e-10;
```

```
    public record Solution(boolean consistent, int rank, double[] values) {}

    /** Solve A*x=b; free variables are set to zero. O(rows*columns*min(rows,columns)). */
    public static Solution solve(double[][] coefficients, double[] rightSide) {
        int rows = coefficients.length;
        if (rightSide.length != rows) throw new IllegalArgumentException("dimension mismatch");
        int columns = rows == 0 ? 0 : coefficients[0].length;
        double[][] matrix = new double[rows][columns + 1];
        for (int row = 0; row < rows; row++) {
            if (coefficients[row].length != columns) throw new IllegalArgumentException("ragged matrix");
            System.arraycopy(coefficients[row], 0, matrix[row], 0, columns);
            matrix[row][columns] = rightSide[row];
        }
        int[] pivotColumn = new int[Math.min(rows, columns)];
        int rank = 0;
        for (int column = 0; column < columns && rank < rows; column++) {
            int pivot = rank;
            for (int row = rank + 1; row < rows; row++)
                if (Math.abs(matrix[row][column]) > Math.abs(matrix[pivot][column])) pivot = row;
            if (Math.abs(matrix[pivot][column]) < EPS) continue;
            double[] swap = matrix[rank]; matrix[rank] = matrix[pivot]; matrix[pivot] = swap;
            double divisor = matrix[rank][column];
            for (int c = column; c <= columns; c++) matrix[rank][c] /= divisor;
            for (int row = 0; row < rows; row++) if (row != rank) {
                double factor = matrix[row][column];
                for (int c = column; c <= columns; c++) matrix[row][c] -= factor * matrix[rank][c];
            }
            pivotColumn[rank] = column;
            rank++;
        }
        for (int row = rank; row < rows; row++) if (Math.abs(matrix[row][columns]) > EPS)
            return new Solution(false, rank, new double[0]);
        double[] answer = new double[columns];
        for (int row = 0; row < rank; row++) answer[pivotColumn[row]] = matrix[row][columns];
        return new Solution(true, rank, answer);
    }

    /** Determinant of a square matrix by elimination. O(n^3). */
    public static double determinant(double[][] input) {
        int n = input.length;
        double[][] matrix = new double[n][n];
        for (int row = 0; row < n; row++) {
            if (input[row].length != n) throw new IllegalArgumentException("matrix must be square");
            matrix[row] = input[row].clone();
        }
        double determinant = 1;
        for (int column = 0; column < n; column++) {
            int pivot = column;
            for (int row = column + 1; row < n; row++)
                if (Math.abs(matrix[row][column]) > Math.abs(matrix[pivot][column])) pivot = row;
            if (Math.abs(matrix[pivot][column]) < EPS) return 0;
            if (pivot != column) { double[] swap = matrix[column]; matrix[column] = matrix[pivot]; matrix[pivot] =
     swap; determinant = -determinant; }
            double value = matrix[column][column];
            determinant *= value;
            for (int row = column + 1; row < n; row++) {
                double factor = matrix[row][column] / value;
                for (int c = column + 1; c < n; c++) matrix[row][c] -= factor * matrix[column][c];
            }
        }
        return determinant;
    }
}
```

10.2 Subset zeta, Mobius, and XOR transforms

Level Advanced
Use when Aggregate values over all submasks or XOR combinations.
Cost O(k*2^k) for k bits.
Watch for Array length must be a power of two; long sums may overflow.

Core idea. Each bit doubles the group of masks that contributes to an aggregate.
Worked example. For f[0]=1 and f[1]=2, zeta gives [1,3].
Practice. Compare every transformed mask with direct submask enumeration.

```
package notebook;

/** Zeta, Mobius, and Walsh-Hadamard transforms on subset masks. */
public final class SubsetTransforms {
    private SubsetTransforms() {}

    private static int bits(long[] values) {
        int length = values.length;
        if (length == 0 || (length & (length - 1)) != 0)
            throw new IllegalArgumentException("length must be a power of two");
        return Integer.numberOfTrailingZeros(length);
    }

    /** Replace f[mask] by the sum of f[submask] over all submasks. O(k*2^k). */
    public static void subsetZeta(long[] values) {
        int k = bits(values);
        for (int bit = 0; bit < k; bit++)
            for (int mask = 0; mask < values.length; mask++) if ((mask & (1 << bit)) != 0)
                values[mask] = Math.addExact(values[mask], values[mask ^ (1 << bit)]);
    }

    /** Undo subsetZeta. O(k*2^k). */
    public static void subsetMobius(long[] values) {
        int k = bits(values);
        for (int bit = 0; bit < k; bit++)
            for (int mask = 0; mask < values.length; mask++) if ((mask & (1 << bit)) != 0)
                values[mask] = Math.subtractExact(values[mask], values[mask ^ (1 << bit)]);
    }

    /** XOR Walsh-Hadamard transform; inverse divides each result by the array length. O(k*2^k). */
    public static void xorHadamard(long[] values, boolean inverse) {
        bits(values);
        for (int width = 1; width < values.length; width <<= 1)
            for (int start = 0; start < values.length; start += width * 2)
                for (int i = 0; i < width; i++) {
                    long a = values[start + i], b = values[start + width + i];
                    values[start + i] = Math.addExact(a, b);
                    values[start + width + i] = Math.subtractExact(a, b);
                }
        if (inverse) for (int i = 0; i < values.length; i++) {
            if (values[i] % values.length != 0) throw new ArithmeticException("nonintegral inverse");
            values[i] /= values.length;
        }
    }
}
```

10.3 Fast Fourier transform convolution

Level Advanced
Use when Multiply long coefficient arrays when floating-point rounding is safe.
Cost O(n log n) for a power-of-two padded length.
Watch for Floating-point rounding is approximate; use NTT for exact modular answers.

Core idea. The FFT turns polynomial multiplication into pointwise multiplication in frequency space.
Worked example. The convolution of [1,2] and [3,4] is [3,10,8].
Practice. Compare with quadratic multiplication using small coefficients.

```
package notebook;

/** Floating-point FFT convolution for integer coefficient sequences. */
public final class FastFourierTransform {
    private FastFourierTransform() {}

    /** Rounded convolution. Use NTT for exact modular arithmetic; floating error grows with input size. */
    public static long[] convolution(long[] first, long[] second) {
        if (first.length == 0 || second.length == 0) return new long[0];
        int resultLength = Math.addExact(first.length, second.length) - 1;
        int size = 1;
        while (size < resultLength) {
            if (size > (1 << 29)) throw new IllegalArgumentException("transform too large");
            size <<= 1;
        }
        double[] realA = new double[size], imagA = new double[size];
        double[] realB = new double[size], imagB = new double[size];
        for (int i = 0; i < first.length; i++) realA[i] = first[i];
        for (int i = 0; i < second.length; i++) realB[i] = second[i];
        transform(realA, imagA, false);
        transform(realB, imagB, false);
        for (int i = 0; i < size; i++) {
            double real = realA[i] * realB[i] - imagA[i] * imagB[i];
            double imag = realA[i] * imagB[i] + imagA[i] * realB[i];
            realA[i] = real; imagA[i] = imag;
        }
        transform(realA, imagA, true);
        long[] result = new long[resultLength];
        for (int i = 0; i < resultLength; i++) {
            double value = Math.rint(realA[i]);
            if (!Double.isFinite(value) || value < -0x1.0p63 || value >= 0x1.0p63)
                throw new ArithmeticException("coefficient outside long range");
            result[i] = (long) value;
        }
        return result;
    }

    private static void transform(double[] real, double[] imag, boolean inverse) {
        int n = real.length;
        for (int i = 1, j = 0; i < n; i++) {
            int bit = n >>> 1;
            for (; (j & bit) != 0; bit >>>= 1) j ^= bit;
            j ^= bit;
            if (i < j) {
                double temp = real[i]; real[i] = real[j]; real[j] = temp;
                temp = imag[i]; imag[i] = imag[j]; imag[j] = temp;
            }
        }
        for (int length = 2; length <= n; length <<= 1) {
            double angle = 2 * Math.PI / length * (inverse ? -1 : 1);
            double rootReal = Math.cos(angle), rootImag = Math.sin(angle);
            for (int start = 0; start < n; start += length) {
                double factorReal = 1, factorImag = 0;
                for (int offset = 0; offset < length / 2; offset++) {
                    int even = start + offset, odd = even + length / 2;
                    double oddReal = real[odd] * factorReal - imag[odd] * factorImag;
                    double oddImag = real[odd] * factorImag + imag[odd] * factorReal;
                    real[odd] = real[even] - oddReal; imag[odd] = imag[even] - oddImag;
                    real[even] += oddReal; imag[even] += oddImag;
```

```
                    double nextReal = factorReal * rootReal - factorImag * rootImag;
                    factorImag = factorReal * rootImag + factorImag * rootReal;
                    factorReal = nextReal;
                }
            }
            if (length == n) break;
        }
        if (inverse) for (int i = 0; i < n; i++) { real[i] /= n; imag[i] /= n; }
    }
}
```

11 Geometry

Use floating-point distances and exact integer predicates as appropriate.

11.1 Distance, polygons, hull, closest pair

Level Intermediate
Use when Measure and optimize among planar points.
Cost O(n) to O(n log^2 n).
Watch for Floating-point orientation is unstable near collinearity.

Core idea. Sort and orient points carefully; choose exact predicates when coordinates are integral.
Worked example. A convex hull removes interior points before farthest-pair search.
Practice. Test duplicate and collinear points.

```
package notebook;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

/** Planar geometry with double coordinates; use exact arithmetic for integer-critical predicates. */
public final class Geometry {
    private Geometry() {}

    public record Point(double x, double y) {}

    public static double distance(Point a, Point b) {
        return Math.hypot(a.x - b.x, a.y - b.y);
    }

    /** Shoelace area; input vertices are not repeated at the end. O(n). */
    public static double polygonArea(List<Point> p) {
        double twice = 0;
        for (int i = 0; i < p.size(); i++) {
            Point a = p.get(i), b = p.get((i + 1) % p.size());
            twice += a.x * b.y - a.y * b.x;
        }
        return Math.abs(twice) / 2;
    }

    /** Closed polygon perimeter; O(n). */
    public static double polygonPerimeter(List<Point> p) {
        double result = 0;
        for (int i = 0; i < p.size(); i++) result += distance(p.get(i), p.get((i + 1) % p.size()));
        return result;
    }

    private static double cross(Point a, Point b, Point c) {
        return (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
    }
```

```
    /** Monotone-chain hull in counterclockwise order; removes collinear interior points. O(n log n). */
    public static List<Point> convexHull(Point[] points) {
        Point[] sorted = points.clone();
        Arrays.sort(sorted, Comparator.comparingDouble(Point::x).thenComparingDouble(Point::y));
        List<Point> unique = new ArrayList<>();
        for (Point p : sorted) if (unique.isEmpty() || !unique.get(unique.size() - 1).equals(p)) unique.add(p);
        if (unique.size() <= 1) return unique;
        List<Point> lower = new ArrayList<>(), upper = new ArrayList<>();
        for (Point p : unique) {
            while (lower.size() >= 2 && cross(lower.get(lower.size() - 2), lower.get(lower.size() - 1), p) <= 0)
                lower.remove(lower.size() - 1);
            lower.add(p);
        }
        for (int i = unique.size() - 1; i >= 0; i--) {
            Point p = unique.get(i);
            while (upper.size() >= 2 && cross(upper.get(upper.size() - 2), upper.get(upper.size() - 1), p) <= 0)
                upper.remove(upper.size() - 1);
            upper.add(p);
        }
        lower.remove(lower.size() - 1);
        upper.remove(upper.size() - 1);
        lower.addAll(upper);
        return lower;
    }

    /** Closest pair distance. O(n log^2 n); requires at least two points. */
    public static double closestPair(Point[] points) {
        if (points.length < 2) throw new IllegalArgumentException("at least two points required");
        Point[] sorted = points.clone();
        Arrays.sort(sorted, Comparator.comparingDouble(Point::x).thenComparingDouble(Point::y));
        return closest(sorted, 0, sorted.length);
    }

    private static double closest(Point[] p, int lo, int hi) {
        if (hi - lo <= 3) {
            double best = Double.POSITIVE_INFINITY;
            for (int i = lo; i < hi; i++) for (int j = i + 1; j < hi; j++) best = Math.min(best, distance(p[i],
     p[j]));
            return best;
        }
        int mid = (lo + hi) >>> 1;
        double splitX = p[mid].x;
        double best = Math.min(closest(p, lo, mid), closest(p, mid, hi));
        List<Point> strip = new ArrayList<>();
        for (int i = lo; i < hi; i++) if (Math.abs(p[i].x - splitX) < best) strip.add(p[i]);
        strip.sort(Comparator.comparingDouble(Point::y));
        for (int i = 0; i < strip.size(); i++)
            for (int j = i + 1; j < strip.size() && strip.get(j).y - strip.get(i).y < best; j++)
                best = Math.min(best, distance(strip.get(i), strip.get(j)));
        return best;
    }

    /** Diameter via convex hull and rotating calipers. O(n log n). */
    public static double farthestPair(Point[] points) {
        List<Point> hull = convexHull(points);
        int n = hull.size();
        if (n < 2) return 0;
        if (n == 2) return distance(hull.get(0), hull.get(1));
        int j = 1;
        double best = 0;
        for (int i = 0; i < n; i++) {
            int next = (i + 1) % n;
            while (Math.abs(cross(hull.get(i), hull.get(next), hull.get((j + 1) % n)))
                    > Math.abs(cross(hull.get(i), hull.get(next), hull.get(j)))) j = (j + 1) % n;
            best = Math.max(best, Math.max(distance(hull.get(i), hull.get(j)), distance(hull.get(next),
     hull.get(j))));
        }
        return best;
    }
}
```

11.2 Exact orientation and intersection

Level Advanced
Use when Need robust integer geometry predicates.
Cost O(1) predicates; O(n) polygon queries.
Watch for BigInteger avoids overflow but allocates.

Core idea. Compute cross products with BigInteger so integer orientation never silently overflows.
Worked example. The sign of the cross product distinguishes left turn, right turn, and collinearity.
Practice. Test segments sharing an endpoint and nearly collinear large coordinates.

```
package notebook;

import java.math.BigInteger;
import java.util.List;

/** Exact integer predicates for planar geometry. */
public final class GeometryAdvanced {
    private GeometryAdvanced() {}

    public record Point(long x, long y) {}

    /** Signed cross product (b-a) x (c-a), exact even near long limits. */
    public static BigInteger cross(Point a, Point b, Point c) {
        BigInteger dx1 = BigInteger.valueOf(b.x).subtract(BigInteger.valueOf(a.x));
        BigInteger dy1 = BigInteger.valueOf(b.y).subtract(BigInteger.valueOf(a.y));
        BigInteger dx2 = BigInteger.valueOf(c.x).subtract(BigInteger.valueOf(a.x));
        BigInteger dy2 = BigInteger.valueOf(c.y).subtract(BigInteger.valueOf(a.y));
        return dx1.multiply(dy2).subtract(dy1.multiply(dx2));
    }

    /** -1 clockwise, 0 collinear, +1 counterclockwise. */
    public static int orientation(Point a, Point b, Point c) {
        return cross(a, b, c).signum();
    }

    private static boolean onSegment(Point a, Point b, Point p) {
        return orientation(a, b, p) == 0 && Math.min(a.x, b.x) <= p.x && p.x <= Math.max(a.x, b.x)
            && Math.min(a.y, b.y) <= p.y && p.y <= Math.max(a.y, b.y);
    }

    /** Inclusive segment intersection; handles endpoints and collinearity. */
    public static boolean segmentsIntersect(Point a, Point b, Point c, Point d) {
        int abC = orientation(a, b, c), abD = orientation(a, b, d);
        int cdA = orientation(c, d, a), cdB = orientation(c, d, b);
        if (abC == 0 && onSegment(a, b, c)) return true;
        if (abD == 0 && onSegment(a, b, d)) return true;
        if (cdA == 0 && onSegment(c, d, a)) return true;
        if (cdB == 0 && onSegment(c, d, b)) return true;
        return abC != abD && cdA != cdB;
    }

    /** True inside or on the boundary of a simple polygon. O(n). */
    public static boolean pointInPolygon(List<Point> polygon, Point point) {
        boolean inside = false;
        for (int i = 0; i < polygon.size(); i++) {
            Point a = polygon.get(i), b = polygon.get((i + 1) % polygon.size());
            if (onSegment(a, b, point)) return true;
            boolean crosses = (a.y > point.y) != (b.y > point.y);
            if (crosses) {
                int sign = orientation(a, b, point);
                if ((b.y > a.y && sign > 0) || (b.y < a.y && sign < 0)) inside = !inside;
            }
        }
        return inside;
    }

    /** Twice the signed polygon area, exact. Input need not repeat the first point. */
    public static BigInteger signedDoubleArea(List<Point> polygon) {
        BigInteger result = BigInteger.ZERO;
```

```
        for (int i = 0; i < polygon.size(); i++) {
            Point a = polygon.get(i), b = polygon.get((i + 1) % polygon.size());
            result = result.add(BigInteger.valueOf(a.x).multiply(BigInteger.valueOf(b.y))
                .subtract(BigInteger.valueOf(a.y).multiply(BigInteger.valueOf(b.x))));
        }
        return result;
    }
}
```

11.3 Lines, segments, and circle intersections

Level Intermediate
Use when Measure distances or intersect lines and circles.
Cost O(1) per primitive.
Watch for Coincident circles have infinitely many intersections.

Core idea. Projection and the common chord solve these primitives.
Worked example. Unit circles centered at (0,0) and (2,0) touch at (1,0).
Practice. Test tangent, disjoint, and degenerate cases.

```
package notebook;

import java.util.List;

/** Floating-point line, segment, and circle primitives with an absolute epsilon. */
public final class CircleGeometry {
    private CircleGeometry() {}
    private static final double EPS = 1e-10;
    public record Point(double x, double y) {}

    /** Intersection of two infinite lines, or null when parallel/coincident. O(1). */
    public static Point lineIntersection(Point a, Point b, Point c, Point d) {
        double ux = b.x - a.x, uy = b.y - a.y;
        double vx = d.x - c.x, vy = d.y - c.y;
        double determinant = ux * vy - uy * vx;
        if (Math.abs(determinant) < EPS) return null;
        double t = ((c.x - a.x) * vy - (c.y - a.y) * vx) / determinant;
        return new Point(a.x + t * ux, a.y + t * uy);
    }

    /** Euclidean distance from a point to a closed segment. O(1). */
    public static double pointSegmentDistance(Point point, Point a, Point b) {
        double dx = b.x - a.x, dy = b.y - a.y;
        double squared = dx * dx + dy * dy;
        if (squared == 0) return Math.hypot(point.x - a.x, point.y - a.y);
        double t = Math.max(0, Math.min(1, ((point.x - a.x) * dx + (point.y - a.y) * dy) / squared));
        return Math.hypot(point.x - (a.x + t * dx), point.y - (a.y + t * dy));
    }

    /** Intersections of an infinite line with a circle. Returns zero, one, or two points. */
    public static List<Point> circleLine(Point center, double radius, Point a, Point b) {
        if (radius < 0) throw new IllegalArgumentException("negative radius");
        double dx = b.x - a.x, dy = b.y - a.y;
        double lengthSquared = dx * dx + dy * dy;
        if (lengthSquared == 0) throw new IllegalArgumentException("line needs two distinct points");
        double t = ((center.x - a.x) * dx + (center.y - a.y) * dy) / lengthSquared;
        Point foot = new Point(a.x + t * dx, a.y + t * dy);
        double offsetSquared = radius * radius - Math.pow(foot.x - center.x, 2) - Math.pow(foot.y - center.y, 2);
        if (offsetSquared < -EPS) return List.of();
        if (Math.abs(offsetSquared) <= EPS) return List.of(foot);
        double offset = Math.sqrt(offsetSquared / lengthSquared);
        return List.of(new Point(foot.x + offset * dx, foot.y + offset * dy),
                new Point(foot.x - offset * dx, foot.y - offset * dy));
    }

    /** Intersections of two circles; coincident circles are rejected. */
    public static List<Point> circleCircle(Point first, double firstRadius, Point second, double secondRadius) {
        if (firstRadius < 0 || secondRadius < 0) throw new IllegalArgumentException("negative radius");
```

```
        double dx = second.x - first.x, dy = second.y - first.y;
        double distance = Math.hypot(dx, dy);
        if (distance < EPS) {
            if (Math.abs(firstRadius - secondRadius) < EPS)
                throw new IllegalArgumentException("infinitely many intersections");
            return List.of();
        }
        if (distance > firstRadius + secondRadius + EPS
                || distance < Math.abs(firstRadius - secondRadius) - EPS) return List.of();
        double along = (firstRadius * firstRadius - secondRadius * secondRadius + distance * distance)
                / (2 * distance);
        double heightSquared = firstRadius * firstRadius - along * along;
        if (heightSquared < -EPS) return List.of();
        double x = first.x + along * dx / distance, y = first.y + along * dy / distance;
        if (Math.abs(heightSquared) <= EPS) return List.of(new Point(x, y));
        double height = Math.sqrt(heightSquared);
        return List.of(new Point(x - height * dy / distance, y + height * dx / distance),
                new Point(x + height * dy / distance, y - height * dx / distance));
    }
}
```

11.4 Two-dimensional KD-tree nearest neighbor

Level Advanced
Use when Find the nearest point repeatedly in a static 2D point set.
Cost O(n log^2 n) build; average O(log n), worst O(n) per query.
Watch for Worst-case query is linear and squared distances can overflow double.

Core idea. Alternating coordinate partitions create boxes that can be pruned by distance lower bounds.
Worked example. For points (0,0) and (4,0), query (3,0) returns the second point.
Practice. Compare random queries with a full scan and include duplicate points.

```
package notebook;

import java.util.Arrays;
import java.util.Comparator;

/** Static two-dimensional nearest-neighbor index using a bounding-box KD tree. */
public final class KDTree2D {
    public record Point(double x, double y) {}
    public record Neighbor(int index, double squaredDistance) {}
    private record Item(double x, double y, int index) {}
    private static final class Node {
        Item item;
        Node left, right;
        double minX, maxX, minY, maxY;
        Node(Item item) { this.item = item; minX=maxX=item.x; minY=maxY=item.y; }
    }
    private final Node root;

    /** Build in O(n log^2 n) time. Points are copied and must have finite coordinates. */
    public KDTree2D(Point[] points) {
        Item[] items = new Item[points.length];
        for (int i = 0; i < points.length; i++) {
            Point point = points[i];
            if (!Double.isFinite(point.x) || !Double.isFinite(point.y))
                throw new IllegalArgumentException("coordinates must be finite");
            items[i] = new Item(point.x, point.y, i);
        }
        root = build(items, 0, items.length, 0);
    }

    private static Node build(Item[] points, int left, int right, int depth) {
        if (left >= right) return null;
        Comparator<Item> order = depth % 2 == 0
                ? Comparator.comparingDouble(Item::x).thenComparingDouble(Item::y).thenComparingInt(Item::index)
                : Comparator.comparingDouble(Item::y).thenComparingDouble(Item::x).thenComparingInt(Item::index);
        Arrays.sort(points, left, right, order);
```

```
        int middle = (left + right) >>> 1;
        Node node = new Node(points[middle]);
        node.left = build(points, left, middle, depth + 1);
        node.right = build(points, middle + 1, right, depth + 1);
        include(node, node.left); include(node, node.right);
        return node;
    }

    private static void include(Node node, Node child) {
        if (child == null) return;
        node.minX=Math.min(node.minX,child.minX); node.maxX=Math.max(node.maxX,child.maxX);
        node.minY=Math.min(node.minY,child.minY); node.maxY=Math.max(node.maxY,child.maxY);
    }

    /** Return original input index and squared distance. Average O(log n), worst O(n). */
    public Neighbor nearest(Point query) {
        if (root == null) throw new IllegalStateException("empty point set");
        if (!Double.isFinite(query.x) || !Double.isFinite(query.y))
            throw new IllegalArgumentException("query must be finite");
        Best best = new Best();
        search(root, query, best);
        return new Neighbor(best.index, best.distance);
    }

    private static final class Best { int index = -1; double distance = Double.POSITIVE_INFINITY; }

    private static void search(Node node, Point query, Best best) {
        if (node == null || boxDistance(node, query) > best.distance) return;
        double dx = node.item.x-query.x, dy = node.item.y-query.y, distance = dx*dx+dy*dy;
        if (distance < best.distance || distance == best.distance && node.item.index < best.index) {
            best.distance=distance; best.index=node.item.index;
        }
        double leftDistance = node.left == null ? Double.POSITIVE_INFINITY : boxDistance(node.left,query);
        double rightDistance = node.right == null ? Double.POSITIVE_INFINITY : boxDistance(node.right,query);
        if (leftDistance <= rightDistance) {
            search(node.left,query,best); search(node.right,query,best);
        } else {
            search(node.right,query,best); search(node.left,query,best);
        }
    }

    private static double boxDistance(Node node, Point query) {
        double dx = query.x < node.minX ? node.minX-query.x : query.x > node.maxX ? query.x-node.maxX : 0;
        double dy = query.y < node.minY ? node.minY-query.y : query.y > node.maxY ? query.y-node.maxY : 0;
        return dx*dx+dy*dy;
    }
}
```

11.5 3D vectors and spherical distance

Level Intermediate
Use when Compute vector products, tetrahedron volume, or distance on a sphere.
Cost O(1) per operation.
Watch for Latitude uses degrees; spherical distance assumes a mean Earth radius.

Core idea. Dot and cross products extend orientation and area calculations to three dimensions.
Worked example. The cross product of (1,0,0) and (0,1,0) is (0,0,1).
Practice. Check orthogonality with dot products and compare great-circle distances with known locations.

```
package notebook;

/** Three-dimensional vector operations and spherical distance. */
public final class Geometry3D {
    private Geometry3D() {}
    public record Vector(double x, double y, double z) {
        public Vector add(Vector other) { return new Vector(x+other.x,y+other.y,z+other.z); }
        public Vector subtract(Vector other) { return new Vector(x-other.x,y-other.y,z-other.z); }
        public Vector scale(double factor) { return new Vector(x*factor,y*factor,z*factor); }
```

```
        public double dot(Vector other) { return x*other.x+y*other.y+z*other.z; }
        public Vector cross(Vector other) {
            return new Vector(y*other.z-z*other.y,z*other.x-x*other.z,x*other.y-y*other.x);
        }
        public double norm() { return Math.sqrt(dot(this)); }
        public double distance(Vector other) { return subtract(other).norm(); }
    }

    /** Signed volume of tetrahedron (origin,a,b,c). */
    public static double signedTetrahedronVolume(Vector a, Vector b, Vector c) {
        return a.dot(b.cross(c))/6.0;
    }

    /** Great-circle distance in kilometers using the haversine formula. */
    public static double greatCircleKilometers(double latitude1, double longitude1,
                                               double latitude2, double longitude2) {
        if (Math.abs(latitude1)>90 || Math.abs(latitude2)>90) throw new IllegalArgumentException("invalid latitude");
        double lat1=Math.toRadians(latitude1), lat2=Math.toRadians(latitude2);
        double dLat=lat2-lat1, dLon=Math.toRadians(longitude2-longitude1);
        double haversine=Math.sin(dLat/2)*Math.sin(dLat/2)
                +Math.cos(lat1)*Math.cos(lat2)*Math.sin(dLon/2)*Math.sin(dLon/2);
        return 6371.0088*2*Math.asin(Math.sqrt(Math.max(0,Math.min(1,haversine))));
    }
}
```

12 Game theory

Model legal moves and terminal states before applying xor or mex.

12.1 Nim, mex, Grundy, minimax

Level Intermediate
Use when Analyze impartial games or complete binary game trees.
Cost O(piles), O(edges), or O(leaves).
Watch for Nim assumes normal play.

Core idea. For impartial normal-play games, a zero xor of Grundy values is losing.
Worked example. Nim heaps of sizes 1, 2, and 3 have xor zero.
Practice. Enumerate all moves in a tiny game to verify Grundy labels.

```
package notebook;

import java.util.HashSet;
import java.util.Set;

public final class GameTheory {
    private GameTheory() {}

    /** Normal-play Nim: first player wins iff pile xor is nonzero. */
    public static boolean firstWinsNim(int[] piles) {
        int xor = 0;
        for (int pile : piles) {
            if (pile < 0) throw new IllegalArgumentException();
            xor ^= pile;
        }
        return xor != 0;
    }

    /** Minimum excluded nonnegative integer. */
    public static int mex(Set<Integer> values) {
        int result = 0;
        while (values.contains(result)) result++;
        return result;
```

```
    }

    /** Grundy values for a DAG whose edges only lead to lower-numbered states. */
    public static int[] grundy(int[][] moves) {
        int[] result = new int[moves.length];
        for (int i = 0; i < moves.length; i++) {
            Set<Integer> reachable = new HashSet<>();
            for (int to : moves[i]) {
                if (to < 0 || to >= i) throw new IllegalArgumentException("moves must lead to lower states");
                reachable.add(result[to]);
            }
            result[i] = mex(reachable);
        }
        return result;
    }

    /** Minimax of a complete binary game tree; leaves length must be a power of two. */
    public static int minimax(int[] leaves) {
        if (leaves.length == 0 || (leaves.length & (leaves.length - 1)) != 0)
            throw new IllegalArgumentException("leaf count must be a power of two");
        int[] level = leaves.clone();
        boolean maximize = (Integer.numberOfTrailingZeros(leaves.length) & 1) == 1;
        while (level.length > 1) {
            int[] next = new int[level.length / 2];
            for (int i = 0; i < next.length; i++)
                next[i] = maximize ? Math.max(level[2 * i], level[2 * i + 1])
                    : Math.min(level[2 * i], level[2 * i + 1]);
            level = next;
            maximize = !maximize;
        }
        return level[0];
    }
}
```